

Оглавление

Введение	5
1 Анализ предметной области	7
1.1 Описание предметной области	7
1.2 Анализ текущих процессов AS-IS	7
1.3 Выявление проблем предметной области	10
1.4 Анализ существующих решений	10
1.5 Сравнение существующих решений	11
1.6 Пользователи системы	11
1.7 Основные сценарии использования	12
1.8 Вывод	13
2. Требования к приложению	15
2.1. Функциональные требования	15
2.2. Нефункциональные требования	18
2.3. Требования к безопасности.....	19
2.4. Требования к ролям и доступу	21
3. Архитектура клиент-серверного приложения	23
3.1. Обоснование выбора клиент-серверной архитектуры	23
3.2. Общая структура приложения	24
3.3. Взаимодействие компонентов	26
3.4. Применение методологии двенадцати факторов.....	29
4. Выбор программного стека.....	31
4.1. Frontend: React, React Router, Redux Toolkit, Axios, CSS Modules.....	31
4.2. Backend: Node.js, Express.js, Sequelize, JWT, bcryptjs, Swagger	32
4.3. База данных: PostgreSQL и Sequelize	34

4.4. Инфраструктура: Docker, Docker Compose, Nginx	34
4.5. Тестирование: Jest, React Testing Library, Supertest	35
4.6. CI/CD: GitHub Actions и Docker Hub	36
4.7. Хостинг: Render	36
4.8. Система контроля версий: Git.....	37
4.9. Обоснование выбора технологий	37
5. Проектирование базы данных	40
5.1. Основные сущности	40
5.2. Связи между сущностями	41
5.3. Хранение вопросов разных типов	42
5.4. Хранение результатов прохождения	43
5.5. Итоговая структура базы данных	43
6. Разработка клиентской части.....	44
6.1. Структура React-приложения	44
6.2. Маршрутизация страниц.....	45
6.3. Навигация и разделение интерфейса по ролям.....	46
6.4. Главная страница.....	47
6.5. Страница тестов.....	47
6.6. Прохождение теста.....	47
6.7. Конструктор тестов и вопросов.....	48
6.8. Административная панель	48
6.9. Аналитика преподавателя	49
6.10. Работа с состоянием через Redux	49
6.11. Взаимодействие с backend	50
7. Разработка серверной части.....	51

7.1. Структура backend-приложения	51
7.2. REST API	52
7.3. Маршруты авторизации	53
7.4. Работа с тестами и вопросами	53
7.5. Отправка ответов и сохранение результатов	54
7.6. Аналитика преподавателя	55
7.7. Административные маршруты	55
7.8. Авторизация и проверка ролей	55
7.9. Валидация входных данных	56
7.10. Работа с PostgreSQL	56
7.11. Swagger-документация и healthcheck	57
7.12. Основные API-маршруты.....	57
8. Тестирование приложения.....	59
8.1. Регистрация и авторизация	59
8.2. Проверка основных пользовательских сценариев.....	61
8.2.1. Просмотр списка тестов	61
8.2.2. Прохождение теста.....	61
8.2.3. Повтор ошибок	63
8.2.4. Создание и редактирование теста.....	63
8.2.5. Добавление вопросов разных типов	64
8.3. Проверка ролевой модели.....	65
8.4. Тестирование административной панели	66
8.5. Тестирование аналитики преподавателя.....	67
8.6. Модульное и компонентное тестирование клиентской части	69
8.7. Модульное тестирование серверной части	70

8.8. Проверка Swagger и healthcheck	71
8.9. Фаззинг-тестирование API.....	72
8.10. Проверка Docker Compose	74
8.11. CI/CD-тестирование	75
8.12. Итог тестирования.....	76
9. Размещение проекта в Git	77
9.1. Организация веток в репозитории.....	77
9.2. Использование Git в процессе разработки.....	78
9.3. Подключение CI/CD.....	79
9.4. Этапы CI/CD-пайплайна	79
10. Контейнеризация и развёртывание	81
10.1. Dockerfile клиентской части	81
10.2. Dockerfile серверной части	82
10.3. Docker Compose	82
10.4. PostgreSQL-контейнер.....	83
10.5. Nginx как reverse проху.....	83
10.6. Публикация Docker-образов в Docker Hub	84
10.7. Развёртывание на Render.....	84
Заключение	86
Список используемой литературы	88

Введение

В условиях активного развития цифровых технологий и перехода образовательных процессов в онлайн-среду особую значимость приобретают инструменты автоматизации контроля знаний. Одним из наиболее распространённых и эффективных методов оценки уровня подготовки обучающихся являются тестовые системы. Они позволяют оперативно проверять знания, обеспечивают объективность оценки и удобны для масштабирования на большое количество пользователей.

Современные образовательные платформы предъявляют высокие требования к гибкости, интерактивности и доступности подобных систем. В частности, актуальной задачей является создание инструментов, позволяющих преподавателям самостоятельно формировать тестовые задания, управлять структурой тестов, анализировать результаты прохождения и получать отчётность по успеваемости обучающихся.

Целью данной курсовой работы является разработка и развёртывание клиент-серверного приложения «Интерактивный конструктор образовательных тестов», обеспечивающего создание, редактирование и прохождение тестов с поддержкой различных типов вопросов, ролевой модели пользователей, административной панели, аналитики результатов и размещения приложения на хостинге.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области;
- определить требования к системе;
- выбрать архитектуру программного решения и описать её;
- обосновать выбор программного стека для реализации приложения;
- реализовать клиентскую и серверную части системы;
- обеспечить механизм аутентификации и авторизации пользователей;
- реализовать взаимодействие с базой данных;

- реализовать ролевую модель и выполнить валидацию входных данных;
- реализовать функциональность создания, редактирования, удаления и прохождения образовательных тестов;
- реализовать административную панель для управления пользователями, ролями, тестами и системными настройками;
- реализовать аналитику результатов прохождения тестов для преподавателя;
- обеспечить экспорт аналитических данных в форматы PDF и Excel;
- добавить автоматическую документацию REST API с использованием Swagger/OpenAPI;
- реализовать healthcheck-маршрут для проверки работоспособности серверной части;
- провести тестирование приложения, включая модульное, компонентное и фаззинг-тестирование;
- организовать хранение исходного кода в системе контроля версий Git;
- настроить CI/CD-процесс для автоматической проверки проекта и сборки контейнеров;
- выполнить контейнеризацию приложения с использованием Docker;
- разместить приложение на хостинге Render;

Объектом исследования является процесс автоматизации контроля знаний обучающихся с использованием тестовых систем.

Предметом исследования выступают методы и средства разработки веб-приложений, реализующих функциональность создания, прохождения, администрирования и анализа результатов образовательных тестов.

Практическая значимость работы заключается в создании программного продукта, который может быть использован в образовательных учреждениях для организации тестирования и контроля знаний обучающихся.

1 Анализ предметной области

1.1 Описание предметной области

Контроль знаний является важной частью образовательного процесса и используется для оценки уровня усвоения учебного материала обучающимися. Одним из наиболее распространённых способов контроля знаний является тестирование, позволяющее быстро проверять результаты обучения, определять уровень подготовки студентов и выявлять пробелы в изучаемом материале.

Тестирование применяется при проведении текущего, промежуточного и итогового контроля. Использование цифровых технологий позволяет выполнять проверку знаний в электронной форме, автоматизировать обработку результатов и сократить временные затраты преподавателей.

В процессе тестирования участвуют несколько категорий пользователей. Студенты проходят тесты и получают результаты, преподаватели создают и редактируют задания, анализируют успеваемость обучающихся, а администраторы выполняют управление системой и пользователями.

Основной задачей автоматизации предметной области является создание системы, позволяющей упростить процесс подготовки тестов, проведения тестирования, обработки результатов и анализа успеваемости.

1.2 Анализ текущих процессов AS-IS

В настоящее время процесс проведения тестирования может выполняться различными способами: с использованием бумажных материалов, электронных документов или существующих онлайн-платформ. При традиционном подходе преподаватель самостоятельно подготавливает тестовые задания, передаёт их студентам, проверяет ответы и формирует итоговые результаты. При использовании отдельных онлайн-сервисов часть операций автоматизируется, однако процесс часто остаётся распределённым между разными инструментами.

Текущий процесс проведения тестирования включает следующие этапы:

1. Подготовка тестовых заданий преподавателем.
2. Передача заданий студентам.
3. Прохождение тестирования.
4. Проверка ответов.
5. Формирование результатов.
6. Анализ успеваемости.

В данном процессе участвуют студент, преподаватель и администратор. Студент выполняет тестирование, преподаватель подготавливает задания и анализирует результаты, администратор обеспечивает управление пользователями и доступом.

Часть операций в текущем процессе может выполняться вручную: создание и изменение тестовых заданий, обработка результатов, подготовка отчётности, анализ успеваемости и управление пользователями. Это увеличивает трудозатраты и снижает удобство работы с результатами тестирования.

Для отображения текущего процесса и его основных элементов используется контекстная IDEF0-диаграмма.

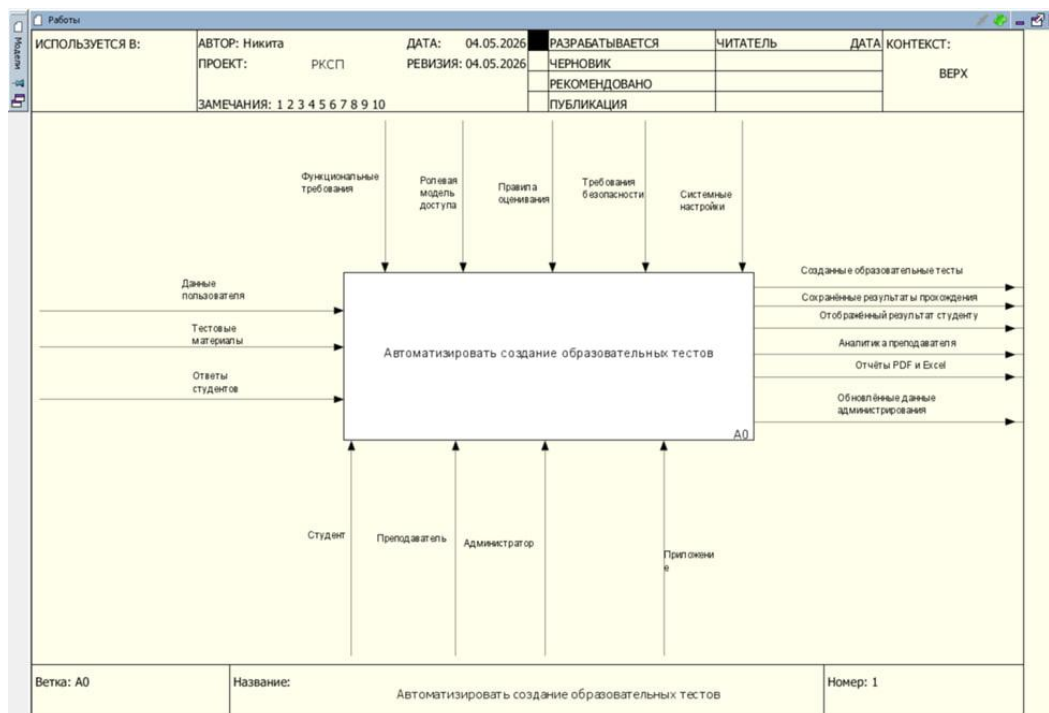


Рисунок 1 — Контекстная IDEF0-диаграмма процесса тестирования

На контекстной диаграмме основным процессом является «Организовать и провести образовательное тестирование».

Входными данными выступают тестовые материалы, данные студентов и ответы обучающихся.

Управляющими воздействиями являются правила оценивания, требования к тестированию, роли пользователей и настройки проведения контроля знаний.

Механизмами выполнения процесса являются студент, преподаватель, администратор и информационная система.

Выходными результатами являются созданные тесты, сохранённые результаты, сведения об успеваемости и отчётные данные.

Для более подробного описания процесса может использоваться декомпозиция IDEF0, отражающая основные этапы работы: создание теста, прохождение тестирования, проверка ответов, сохранение результата и анализ успеваемости.

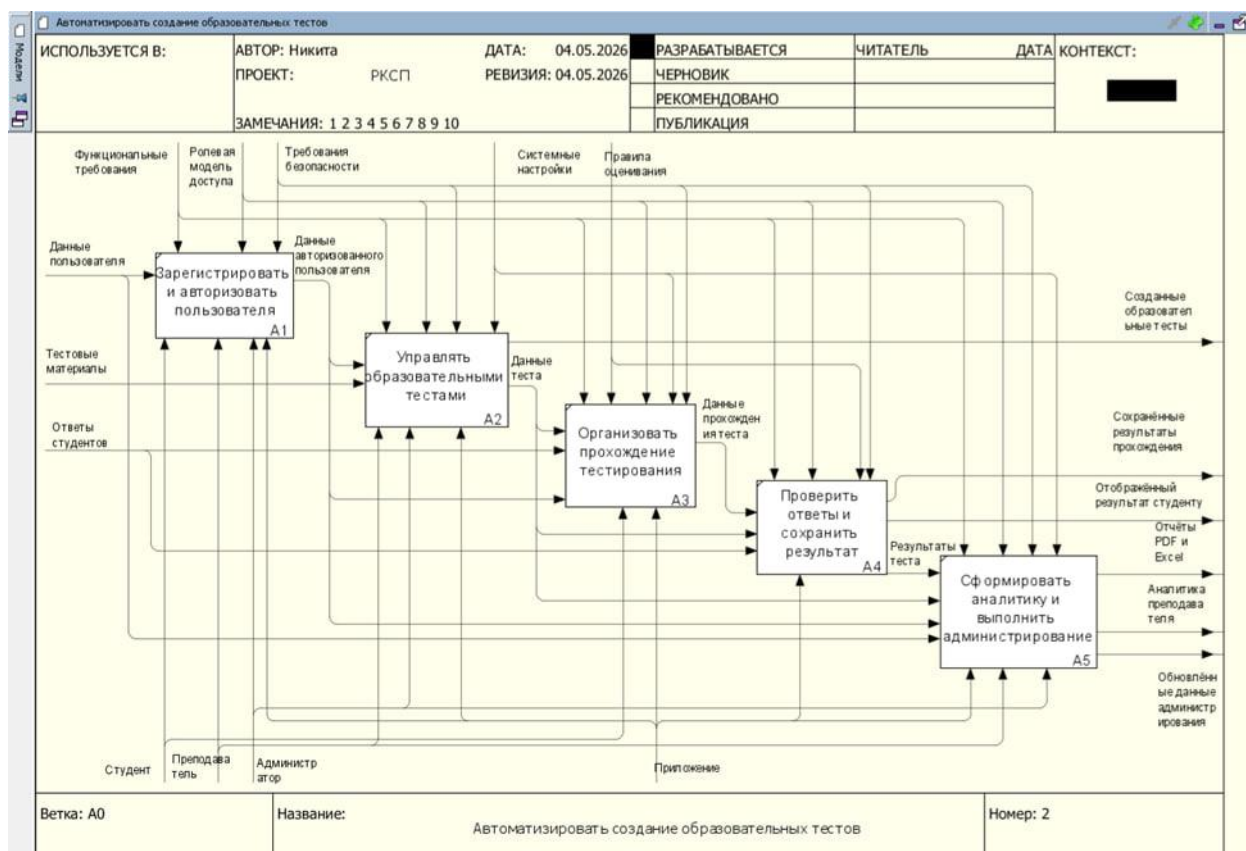


Рисунок 2 — Декомпозиция процесса образовательного тестирования

1.3 Выявление проблем предметной области

В результате анализа текущего процесса были выявлены основные проблемы, возникающие при организации образовательного тестирования.

Таблица 1 — Основные проблемы предметной области

Проблема	Последствие
Ручная обработка результатов	Увеличение времени проверки
Отсутствие централизованного хранения данных	Возможность потери или дублирования информации
Использование разрозненных инструментов	Усложнение работы преподавателя и администратора
Ограниченные возможности анализа	Сложность оценки успеваемости студентов
Недостаточное управление ролями пользователей	Риск доступа к лишним функциям
Зависимость от сторонних сервисов	Ограничение возможностей настройки системы
Отсутствие единого процесса работы с тестами	Сложность сопровождения тестовых материалов

Выявленные недостатки показывают необходимость создания единой информационной системы, объединяющей функции создания тестов, прохождения тестирования, хранения результатов, анализа успеваемости и управления пользователями.

1.4 Анализ существующих решений

Для решения задач онлайн-тестирования существует большое количество программных продуктов и образовательных платформ. К наиболее распространённым решениям относятся Moodle, Google Forms и Quizizz.

Moodle представляет собой систему управления обучением, включающую инструменты создания курсов, тестирования и контроля успеваемости. Преимуществом данного решения является широкий функционал и развитая ролевая модель. Недостатком является высокая сложность настройки и сопровождения, особенно для небольших проектов.

Google Forms позволяет быстро создавать формы и тесты, а также автоматически проверять ответы пользователей. Основными преимуществами являются простота использования и доступность. Недостатком являются ограниченные возможности настройки, слабая ролевая модель и отсутствие полноценного управления образовательным процессом.

Quizizz является платформой для интерактивного тестирования и образовательных заданий. Преимуществами являются удобный интерфейс, интерактивность и наличие готовых материалов. Ограничением является зависимость от внешнего сервиса и невозможность полного изменения внутренней логики работы.

1.5 Сравнение существующих решений

Таблица 2 — Сравнение существующих решений

Критерий	Moodle	Google Forms	Quizizz	Разрабатываемая система
Создание тестов	Да	Да	Да	Да
Поддержка разных типов вопросов	Да	Частично	Да	Да
Прохождение тестов студентами	Да	Да	Да	Да
Автоматическая проверка ответов	Да	Да	Да	Да
Ролевая модель	Да	Нет	Частично	Да
Управление пользователями	Да	Нет	Ограничено	Да
Анализ результатов	Да	Частично	Да	Да
Повтор ошибок	Ограничено	Нет	Частично	Да
Гибкая настройка логики	Ограничено	Нет	Нет	Да
Контроль структуры данных	Частично	Нет	Нет	Да

Проведённый анализ показывает, что существующие решения обладают широкими возможностями, однако не всегда позволяют гибко настраивать процессы, управлять ролями и полностью контролировать данные. Поэтому разработка собственного решения является обоснованной.

1.6 Пользователи системы

В системе предусматриваются три основные категории пользователей: студент, преподаватель и администратор.

Студент использует систему для прохождения тестов, просмотра результатов и повторения ошибок. Его основная задача — выполнить тестирование и получить информацию о своих результатах.

Преподаватель отвечает за создание и сопровождение тестовых материалов. Он создаёт тесты, добавляет вопросы, изменяет задания и анализирует результаты прохождения студентами.

Администратор выполняет управление пользователями и контролирует корректность работы системы. Он может управлять ролями, доступом и основными параметрами функционирования системы.

Распределение пользователей по ролям необходимо для ограничения доступа к функциям и защиты данных. Студент не должен иметь доступ к управлению тестами, преподаватель должен работать с учебными материалами и результатами, а администратор должен выполнять общее управление системой.

1.7 Основные сценарии использования

Основные сценарии использования отражают типовые действия пользователей в системе.

Студент выполняет вход в систему, открывает список доступных тестов, выбирает нужный тест, проходит его и получает результат. После завершения тестирования студент может просмотреть ошибки и повторить проблемные вопросы.

Преподаватель создаёт тест, задаёт его параметры, добавляет вопросы разных типов, редактирует задания и просматривает результаты прохождения. На основе полученных данных преподаватель анализирует успеваемость студентов.

Администратор управляет пользователями, назначает роли, контролирует доступ к функциям и выполняет настройку системы.

Для отображения взаимодействия пользователей с системой используется UML-диаграмма вариантов использования.

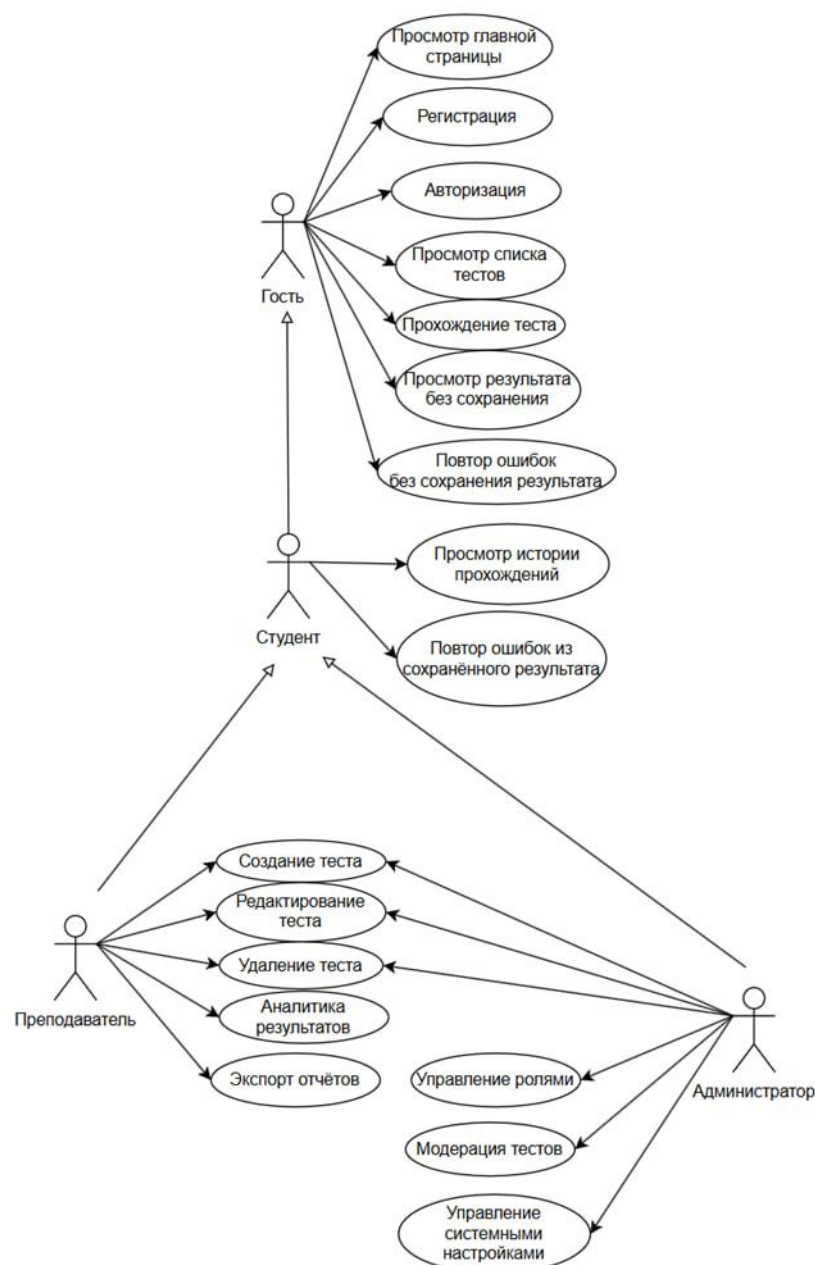


Рисунок 3 — UML-диаграмма вариантов использования

1.8 Вывод

В результате анализа предметной области было установлено, что процесс организации образовательного тестирования требует автоматизации операций, связанных с созданием тестов, прохождением заданий, обработкой результатов и анализом успеваемости.

Анализ текущих процессов показал наличие ручных операций, разрозненного хранения данных и ограниченных возможностей анализа результатов. Рассмотренные существующие решения позволяют частично

решить данные задачи, однако имеют ограничения, связанные со сложностью настройки, зависимостью от сторонних сервисов и недостаточной гибкостью.

Использование IDEF0-диаграммы позволило формализовать процесс образовательного тестирования, определить входные данные, управляющие воздействия, участников процесса и выходные результаты. Полученные результаты анализа являются основой для формирования требований к разрабатываемому приложению.

2. Требования к приложению

Разрабатываемое приложение «Интерактивный конструктор образовательных тестов» должно обеспечивать создание, редактирование, прохождение и администрирование образовательных тестов. При формировании требований учитываются клиент-серверная архитектура, ролевое разграничение доступа, хранение данных в базе PostgreSQL, работа через REST API, возможность контейнеризации, автоматизированного тестирования и размещения приложения на хостинге.

Система должна поддерживать три основные роли пользователей: студент, преподаватель и администратор. Студент использует приложение для прохождения тестов и просмотра результатов. Преподаватель создаёт тесты, управляет вопросами и анализирует результаты прохождения. Администратор управляет пользователями, ролями, публикацией тестов и системными настройками приложения.

Кроме базовой функциональности конструктора тестов, приложение должно поддерживать административную панель, аналитику результатов, экспорт отчётов, проверку работоспособности backend-сервиса через healthcheck-маршрут и документирование API с помощью Swagger/OpenAPI.

2.1. Функциональные требования

Функциональные требования определяют основные возможности приложения, которые должны быть реализованы для пользователей разных ролей.

Система должна обеспечивать регистрацию и авторизацию пользователей. При регистрации пользователь создаёт учётную запись, а при входе получает JWT-токен, который используется для обращения к защищённым маршрутам. По умолчанию пользователь регистрируется с ролью студента.

Студент должен иметь возможность просматривать список доступных тестов, выбирать тест, проходить его, отвечать на вопросы разных типов и получать результат. В приложении поддерживаются вопросы с одним

правильным ответом, с несколькими правильными ответами и на сопоставление. При прохождении теста система должна учитывать ограничение по времени, рассчитывать итоговый результат и сохранять длительность прохождения.

Преподаватель должен иметь возможность создавать тесты, задавать их название, описание и лимит количества вопросов. Также преподаватель должен добавлять, редактировать и удалять вопросы. После прохождения тестов студентами преподаватель должен иметь возможность просматривать аналитику результатов, группировать данные по студентам или тестам, а также экспортировать отчёт в PDF и Excel.

Администратор должен иметь возможность управлять пользователями, изменять их роли, удалять пользователей, просматривать список тестов, включать или отключать активность тестов, а также изменять системные настройки приложения. К системным настройкам относятся название платформы, возможность публичной регистрации и необходимость модерации тестов преподавателей.

Серверная часть должна предоставлять REST API для взаимодействия с клиентской частью, выполнять CRUD-операции, проверять роли пользователей, валидировать входные данные, сохранять результаты прохождения и предоставлять данные для аналитики. В приложении также должны быть доступны Swagger-документация API и healthcheck-маршрут для проверки работоспособности backend-сервиса.

Основные функциональные требования представлены в таблице 2.

Таблица 3 - Функциональные требования к приложению

№	Требование	Описание
1	Регистрация пользователя	Пользователь может создать учётную запись в системе
2	Авторизация пользователя	Пользователь может войти в систему по email и паролю
3	Использование JWT	После входа пользователь получает токен доступа для защищённых запросов
4	Получение профиля	Авторизованный пользователь может получить данные своей учётной записи
5	Просмотр списка тестов	Пользователь может получить список доступных тестов
6	Просмотр теста	Пользователь может открыть выбранный тест

7	Прохождение теста	Студент может отвечать на вопросы и завершать тест
8	Ограничение времени	При прохождении теста учитывается ограничение по времени
9	Автоматический расчёт результата	Система рассчитывает количество правильных ответов и итоговый процент
10	Сохранение результата	Результат прохождения сохраняется в базе данных
11	Сохранение времени прохождения	В результат сохраняется длительность прохождения в секундах
12	Повтор ошибок	Пользователь может повторить вопросы, на которые ранее ответил неправильно
13	Создание теста	Преподаватель или администратор может создать новый тест
14	Редактирование теста	Преподаватель или администратор может изменить название, описание и параметры теста
15	Удаление теста	Преподаватель или администратор может удалить тест
16	Лимит вопросов	При создании или редактировании теста можно задать ограничение количества вопросов
17	Управление активностью теста	Администратор может включать или отключать доступность теста
18	Добавление вопросов	Преподаватель или администратор может добавлять вопросы в тест
19	Редактирование вопросов	Преподаватель или администратор может изменять текст вопроса, варианты и правильный ответ
20	Удаление вопросов	Преподаватель или администратор может удалять вопросы
21	Поддержка вопроса single	Система поддерживает вопрос с одним правильным ответом
22	Поддержка вопроса multiple	Система поддерживает вопрос с несколькими правильными ответами
23	Поддержка вопроса matching	Система поддерживает вопрос на сопоставление элементов
24	Аналитика преподавателя	Преподаватель может просматривать результаты прохождения тестов студентами
25	Группировка аналитики	Результаты могут группироваться по студентам или по тестам
26	Просмотр деталей ответов	Преподаватель может видеть правильные и неправильные ответы студентов
27	Экспорт в PDF	Преподаватель может экспортировать отчёт об успеваемости в PDF
28	Экспорт в Excel	Преподаватель может экспортировать отчёт об успеваемости в Excel
29	Управление пользователями	Администратор может просматривать список пользователей
30	Изменение ролей	Администратор может изменять роль пользователя
31	Удаление пользователей	Администратор может удалять пользователей
32	Модерация тестов	Администратор может управлять активностью опубликованных тестов
33	Управление системными настройками	Администратор может изменять настройки платформы
34	Настройка названия платформы	Администратор может изменить отображаемое название системы

35	Настройка публичной регистрации	Администратор может включить или отключить самостоятельную регистрацию
36	Настройка модерации тестов	Администратор может включить или отключить модерацию тестов преподавателей
37	REST API	Клиентская часть должна взаимодействовать с сервером через API
38	Swagger-документация	Сервер должен предоставлять документацию API
39	Healthcheck	Сервер должен иметь маршрут проверки работоспособности
40	Работа с PostgreSQL	Пользователи, тесты, вопросы, результаты и настройки должны храниться в базе данных

2.2. Нефункциональные требования

Нефункциональные требования определяют характеристики качества приложения и условия его эксплуатации.

Приложение должно иметь удобный и понятный интерфейс, рассчитанный на пользователей с разными ролями. Студент должен быстро находить тесты и проходить их, преподаватель - управлять тестами и анализировать результаты, администратор - выполнять управление системой через отдельную административную панель.

Клиентская часть должна быть реализована как одностраничное React-приложение. Интерфейс должен корректно работать в современных браузерах, поддерживать маршрутизацию между страницами и обновлять состояние без полной перезагрузки страницы.

Серверная часть должна быть отделена от клиентской и предоставлять REST API. Все важные операции должны выполняться через backend, так как именно сервер отвечает за проверку авторизации, ролей, входных данных и сохранение информации в базе данных.

Приложение должно быть переносимым и удобным для развёртывания. Для этого используются Docker и Docker Compose. В проекте также предусмотрены отдельные переменные окружения для локальной и production-среды, включая адрес API, параметры подключения к базе данных, JWT-секрет, срок действия токена и настройки CORS. В full.txt также присутствуют production-переменные для frontend и backend, используемые при размещении на Render.

Система должна быть пригодна для автоматической проверки. Для этого должны использоваться тесты клиентской и серверной части, а также CI/CD-процесс, выполняющий установку зависимостей, запуск тестов, сборку контейнеров и проверку доступности сервисов.

Основные нефункциональные требования представлены в таблице 3.

Таблица 4 - Нефункциональные требования к приложению

№	Требование	Описание
1	Удобство использования	Интерфейс должен быть понятен студенту, преподавателю и администратору
2	Разделение интерфейса по ролям	Пользователь должен видеть только доступные ему разделы
3	Надёжность	Приложение должно корректно обрабатывать ошибки пользователя и сервера
4	Переносимость	Приложение должно запускаться в Docker-контейнерах
5	Масштабируемость	Архитектура должна позволять добавлять новые страницы, маршруты и роли
6	Сопровождаемость	Код должен быть разделён на клиентскую часть, серверную часть, модели, маршруты, контроллеры и middleware
7	Конфигурируемость	Настройки должны задаваться через переменные окружения
8	Совместимость	Приложение должно работать в современных браузерах
9	Производительность	Основные страницы должны загружаться без значительных задержек
10	Устойчивость API	Backend должен возвращать понятные сообщения об ошибках
11	Документируемость	REST API должен быть описан через Swagger/OpenAPI
12	Контроль доступности	Backend должен иметь healthcheck-маршрут
13	Автоматизация проверки	Проект должен поддерживать запуск тестов и проверок через CI/CD
14	Поддержка production-среды	Приложение должно запускаться с production-переменными окружения
15	Развёртывание на хостинге	Frontend и backend должны быть доступны через публичные URL после размещения
16	Хранение данных	Основные данные должны храниться в PostgreSQL
17	Адаптивность интерфейса	Страницы должны корректно отображаться на разных размерах экрана
18	Расширяемость аналитики	Система должна позволять расширять отчёты и способы группировки результатов

2.3. Требования к безопасности

Так как приложение работает с пользовательскими данными, результатами тестирования и административными функциями, необходимо предусмотреть базовые меры безопасности.

Доступ к системе должен осуществляться через регистрацию и авторизацию. При входе пользователь указывает email и пароль, после чего сервер проверяет данные и формирует JWT-токен. Токен используется клиентской частью при обращении к защищённым маршрутам.

Пароли пользователей не должны храниться в открытом виде. Перед сохранением пароль должен хешироваться. При отправке данных пользователя на клиент пароль должен исключаться из ответа.

Доступ к функциональности должен ограничиваться ролью пользователя. Студент не должен иметь доступ к созданию и редактированию тестов. Преподаватель должен иметь доступ к конструктору и аналитике, но не должен управлять пользователями и системными настройками. Администратор должен иметь доступ к административным функциям.

Административные маршруты должны быть защищены авторизацией и проверкой роли admin. В коде для этого используется middleware авторизации и проверки роли, а маршруты /api/admin/... доступны только администратору.

Серверная часть должна выполнять валидацию входных данных. Это относится к email, паролю, имени пользователя, идентификаторам тестов и вопросов, ролям, статусу активности теста и системным настройкам.

Конфиденциальные параметры, такие как строка подключения к базе данных, JWT-секрет и production-настройки, должны храниться в переменных окружения.

Основные требования к безопасности представлены в таблице 4.

Таблица 5 - Требования к безопасности

№	Требование	Описание
1	Аутентификация	Доступ к защищённым функциям осуществляется после входа пользователя
2	Авторизация	Доступ к действиям определяется ролью пользователя
3	JWT-токен	Для подтверждения сессии используется токен доступа
4	Проверка токена	Сервер должен проверять наличие и корректность токена
5	Обработка истёкшего токена	При истечении срока действия токена сервер должен возвращать ошибку авторизации
6	Хеширование паролей	Пароли пользователей не должны храниться в открытом виде
7	Исключение пароля из	При передаче данных пользователя пароль не должен

	ответа	отправляться клиенту
8	Ограничение доступа по ролям	Функции создания, редактирования и удаления должны быть доступны только разрешённым ролям
9	Защита административных маршрутов	Маршруты администратора должны быть доступны только роли admin
10	Валидация входных данных	Сервер должен проверять данные перед обработкой запроса
11	Проверка идентификаторов	id, testId и questionId должны быть положительными целыми числами
12	Проверка ролей	При изменении роли допускаются только значения student, teacher, admin
13	Защита системных настроек	Изменять настройки приложения может только администратор
14	Защита конфигурации	Секретные параметры должны храниться в переменных окружения
15	Ограничение публичной регистрации	Самостоятельная регистрация должна создавать обычного студента, а не администратора
16	Защита от некорректных запросов	Сервер должен возвращать ошибку при неверной структуре данных
17	Контроль CORS	Backend должен принимать запросы только с разрешённых источников в production-среде
18	Безопасное удаление пользователей	Административное удаление пользователей должно выполняться только после проверки прав

2.4. Требования к ролям и доступу

В приложении используется ролевая модель доступа. Она необходима для разделения возможностей пользователей и предотвращения выполнения запрещённых действий.

В модели пользователя предусмотрены три роли: student, teacher и admin. Роль по умолчанию - student. Это означает, что новый пользователь после регистрации получает минимальные права и не имеет доступа к управлению тестами или административной панели. В коде роль пользователя описана как перечисление из трёх значений.

Студент может регистрироваться, входить в систему, просматривать доступные тесты, проходить их, получать результат и повторять ошибки. Он не может создавать, редактировать или удалять тесты и вопросы.

Преподаватель может использовать функциональность студента, а также создавать тесты, редактировать их, добавлять вопросы, удалять вопросы и просматривать аналитику результатов. В интерфейсе для преподавателя отображаются разделы конструктора и аналитики. В тестах клиентской части

также проверяется, что преподавателю доступна навигация к «Конструктору» и «Аналитике».

Администратор имеет доступ к функциям управления системой. Он может управлять пользователями, изменять роли, удалять пользователей, модерировать тесты и изменять системные настройки. В интерфейсе для администратора отображается раздел «Администрирование», а аналитика преподавателя ему не показывается как отдельный пункт навигации.

Распределение прав доступа представлено в таблице 5.

Таблица 6 - Ролевая модель доступа

Действие	Студент	Преподаватель	Администратор
Регистрация	Да	Да	Да
Вход в систему	Да	Да	Да
Получение профиля	Да	Да	Да
Просмотр списка тестов	Да	Да	Да
Просмотр активных тестов	Да	Да	Да
Прохождение тестов	Да	Да	Да
Просмотр результата	Да	Да	Да
Повтор ошибок	Да	Да	Да
Создание тестов	Нет	Да	Да
Редактирование тестов	Нет	Да	Да
Удаление тестов	Нет	Да	Да
Указание лимита вопросов	Нет	Да	Да
Добавление вопросов	Нет	Да	Да
Редактирование вопросов	Нет	Да	Да
Удаление вопросов	Нет	Да	Да
Доступ к конструктору	Нет	Да	Да
Просмотр аналитики результатов	Нет	Да	Нет
Экспорт аналитики в PDF	Нет	Да	Нет
Экспорт аналитики в Excel	Нет	Да	Нет
Просмотр списка пользователей	Нет	Нет	Да
Изменение ролей пользователей	Нет	Нет	Да
Удаление пользователей	Нет	Нет	Да
Просмотр всех тестов для модерации	Нет	Нет	Да
Включение и отключение активности теста	Нет	Нет	Да
Управление системными настройками	Нет	Нет	Да
Изменение названия платформы	Нет	Нет	Да
Включение или отключение публичной регистрации	Нет	Нет	Да
Включение или отключение модерации тестов	Нет	Нет	Да
Доступ к административной панели	Нет	Нет	Да

3. Архитектура клиент-серверного приложения

3.1. Обоснование выбора клиент-серверной архитектуры

Для разработки приложения «Интерактивный конструктор образовательных тестов» выбрана клиент-серверная архитектура, позволяющая разделить пользовательский интерфейс, бизнес-логику, обработку данных и хранение информации.

Клиентская часть отвечает за отображение интерфейса, маршрутизацию страниц, работу с состоянием приложения и отправку запросов к серверу. Пользователь взаимодействует с приложением через браузер: регистрируется, входит в систему, просматривает тесты, проходит тестирование, работает с конструктором, аналитикой или административной панелью в зависимости от своей роли.

Серверная часть отвечает за обработку HTTP-запросов, регистрацию и авторизацию пользователей, проверку JWT-токена, разграничение доступа по ролям, выполнение CRUD-операций, проверку ответов, сохранение результатов, формирование аналитики и управление системными настройками.

База данных PostgreSQL используется для долговременного хранения данных приложения: пользователей, тестов, вопросов, результатов прохождения и системных настроек. Работа с базой данных выполняется серверной частью через модели Sequelize.

Выбор клиент-серверной архитектуры обоснован необходимостью поддержки трёх ролей пользователей: студента, преподавателя и администратора. Такая архитектура позволяет централизованно контролировать доступ к данным и выполнять проверку прав на стороне сервера, а не только в интерфейсе.

Кроме того, клиент-серверная архитектура удобна для контейнеризации и развёртывания. Frontend, backend, PostgreSQL и Nginx могут запускаться как отдельные сервисы в Docker Compose, а в production-среде frontend и backend могут быть размещены как самостоятельные веб-сервисы на хостинге Render.

3.2. Общая структура приложения

Разрабатываемое приложение состоит из следующих основных компонентов:

- React SPA - клиентское одностраничное приложение;
- Redux Store - хранилище состояния клиентского приложения;
- API Service - модуль отправки HTTP-запросов к серверу;
- REST API - серверный программный интерфейс;
- Express App - backend-приложение на Node.js и Express.js;
- PostgreSQL - реляционная база данных;
- Sequelize - ORM для описания моделей и взаимодействия с БД;
- Swagger/OpenAPI - документация серверного API;
- Healthcheck endpoint - маршрут проверки работоспособности backend;
- Nginx - reverse proxy для Docker-среды;
- Docker и Docker Compose - средства контейнеризации и запуска сервисов;
- Render - хостинг для размещения frontend и backend приложения.

Клиентская часть реализована как SPA-приложение на React. Пользователь работает с интерфейсом в браузере, а переходы между страницами выполняются без полной перезагрузки сайта. В приложении предусмотрены страницы главной, регистрации, входа, профиля, списка тестов, прохождения теста, конструктора вопросов, административной панели и аналитики преподавателя.

Маршрутизация выполняется на стороне клиента с использованием React Router. Для хранения состояния авторизации и прохождения тестов используется Redux Toolkit. В клиентской части также выделен API-сервис, через который компоненты и страницы обращаются к серверной части.

Серверная часть реализована на Node.js с использованием Express.js. Она предоставляет REST API для регистрации, авторизации, получения профиля, работы с тестами, вопросами, результатами, аналитикой преподавателя, административными функциями и системными настройками.

В backend-приложении выделены отдельные логические модули:

авторизации, пользователей, тестов, вопросов, результатов, аналитики, административный модуль, модуль системных настроек, middleware авторизации, middleware проверки ролей, middleware валидации входных данных, Swagger-документации, healthcheck-маршрут.

База данных PostgreSQL хранит основные сущности приложения: пользователей, тесты, вопросы, результаты прохождения, параметры ограничения количества вопросов и активности, у результатов сохраняется длительность прохождения.

Nginx используется в Docker-конфигурации как reverse proxy. Он принимает входящие HTTP-запросы, перенаправляет запросы к frontend-сервису, а запросы с префиксом /api/ - к backend-сервису. Также в конфигурации Nginx добавлены базовые заголовки безопасности.

В проекте используются две Docker Compose-конфигурации. Файл docker-compose.yml предназначен для локального запуска и открывает порты PostgreSQL, backend, frontend и Nginx. Файл docker-compose.ubuntu.yml предназначен для серверного запуска: backend и frontend доступны внутри Docker-сети через expose, а внешний доступ выполняется через Nginx.

Приложение размещено на хостинге Render. Frontend доступен как отдельный сервис, backend - как отдельный сервис, а подключение frontend к backend выполняется через переменную окружения REACT_APP_API_URL. Для backend в production-среде используются переменные PORT, NODE_ENV, DATABASE_URL, JWT_SECRET, JWT_EXPIRE, FRONTEND_URL и CORS_ORIGINS.

Общая архитектура приложения может быть представлена следующим образом:

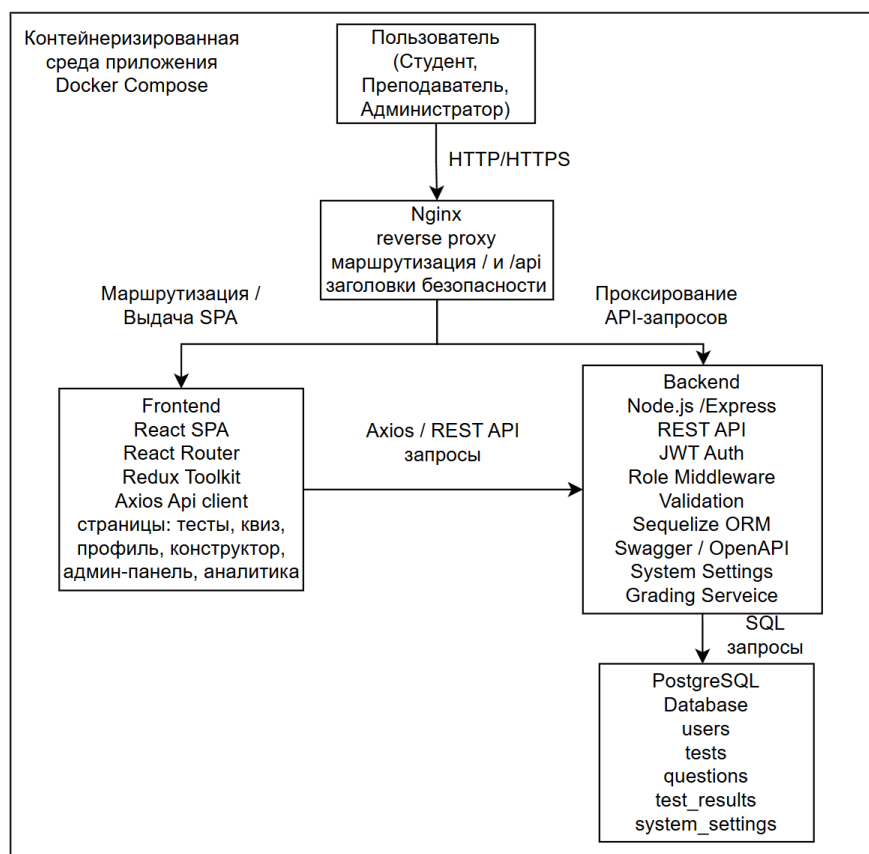


Рисунок 4 - Общая архитектура приложения «Интерактивный конструктор образовательных тестов»

3.3. Взаимодействие компонентов

Взаимодействие компонентов начинается с действий пользователя в браузере. Пользователь открывает приложение, выполняет регистрацию или вход, после чего получает доступ к функциональности в соответствии со своей ролью.

Если пользователь является студентом, он может открыть список активных тестов, выбрать тест, пройти его, отправить ответы и получить результат. Во время прохождения теста клиентская часть хранит текущее состояние тестирования, отображает вопросы, учитывает выбранные ответы и отправляет итоговые данные на сервер. Сервер проверяет ответы, рассчитывает результат, сохраняет его в базе данных и возвращает данные для отображения результата.

Если пользователь является преподавателем, он может открыть конструктор тестов, создать новый тест, задать название, описание и лимит

вопросов, добавить вопросы разных типов, отредактировать или удалить существующие вопросы. Также преподаватель может открыть страницу аналитики, где получает данные о прохождении тестов студентами, просматривает результаты и экспортирует отчёты в PDF или Excel.

Если пользователь является администратором, он может открыть административную панель. Через неё администратор получает список пользователей, изменяет роли, удаляет пользователей, просматривает тесты, включает или отключает активность тестов и изменяет системные настройки приложения.

Клиентская часть отправляет HTTP-запросы к серверному API. Например, при авторизации пользователь вводит email и пароль, после чего frontend передаёт эти данные на сервер. Сервер проверяет данные пользователя в базе данных, сравнивает пароль, формирует JWT-токен и возвращает его клиенту. Далее клиентская часть использует этот токен при обращении к защищённым маршрутам.

При обращении к защищённым маршрутам backend сначала проверяет наличие и корректность JWT-токена. После этого middleware проверки ролей определяет, имеет ли пользователь право выполнить запрошенное действие. Если роль пользователя не входит в список разрешённых, сервер возвращает ошибку доступа. В коде предусмотрена отдельная проверка ролей и валидация входных данных.

Взаимодействие компонентов можно описать следующим образом:

1. Пользователь выполняет действие в интерфейсе.
2. React SPA обрабатывает действие пользователя.
3. Redux Store обновляет состояние приложения, если это необходимо.
4. API Service формирует HTTP-запрос к серверу.
5. В Docker-среде Nginx принимает запрос.
6. Если запрос относится к frontend, Nginx направляет его к frontend-сервису.
7. Если запрос относится к /api/, Nginx направляет его к backend-сервису.

8. Backend проверяет JWT-токен пользователя.
9. Backend проверяет роль пользователя.
10. Backend выполняет валидацию входных данных.
11. Backend выполняет бизнес-логику.
12. Backend обращается к PostgreSQL через Sequelize.
13. PostgreSQL возвращает данные.
14. Backend формирует JSON-ответ.
15. React SPA получает ответ и отображает результат пользователю.

В production-размещении на Render взаимодействие отличается тем, что frontend и backend работают как отдельные публичные сервисы. Пользователь открывает frontend-сервис Render, а клиентская часть отправляет API-запросы на backend-сервис Render по адресу, указанному в REACT_APP_API_URL. Backend подключается к production-базе данных через DATABASE_URL и ограничивает CORS-запросы через CORS_ORIGINS.

Компонентную структуру приложения целесообразно представить с помощью UML-диаграммы компонентов.

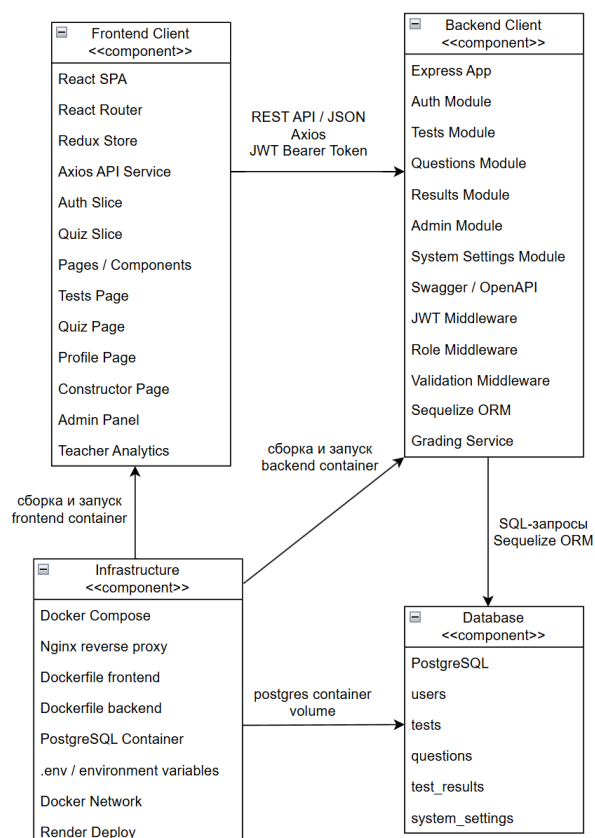


Рисунок 5 - UML-диаграмма компонентов приложения

3.4. Применение методологии двенадцати факторов

При разработке приложения учитываются принципы методологии двенадцати факторов. Данная методология применяется для создания переносимых, масштабируемых и удобных для развёртывания веб-приложений.

В рамках проекта особенно важны следующие принципы.

Кодовая база. Исходный код приложения хранится в системе контроля версий Git. Это позволяет отслеживать изменения, возвращаться к предыдущим версиям и организовать разработку проекта.

Зависимости. Зависимости клиентской и серверной частей явно описаны в файлах `package.json` и устанавливаются отдельно для `frontend` и `backend`. Это обеспечивает воспроизводимую установку приложения на другой машине, сервере или в CI/CD-среде.

Конфигурация. Настройки приложения вынесены во внешние переменные окружения. Для локального запуска используются переменные `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `JWT_SECRET`, `NODE_ENV` и `REACT_APP_API_URL`. Для `production`-среды дополнительно используются `DATABASE_URL`, `JWT_EXPIRE`, `PORT`, `FRONTEND_URL` и `CORS_ORIGINS`.

Сторонние службы. База данных PostgreSQL рассматривается как подключаемый ресурс. Backend взаимодействует с ней через строку подключения `DATABASE_URL`, что позволяет использовать как локальную базу данных в Docker, так и облачную базу данных при размещении на Render.

Процессы. Клиентская часть, серверная часть, база данных и Nginx запускаются как отдельные процессы в отдельных контейнерах. Это повышает изолированность компонентов и упрощает развёртывание.

Сборка, релиз и запуск. Dockerfile и Docker Compose отделяют этап сборки приложения от этапа запуска. Для `frontend` при сборке передаётся аргумент `REACT_APP_API_URL`, что позволяет заранее указать адрес backend-сервиса для `production`-сборки.

Порт привязки. Backend-сервис запускается на порту 5000, frontend в Docker-среде работает на порту 80, а локально доступен через порт 3001. Nginx принимает внешние запросы на порту 80 и перенаправляет их к нужному сервису.

Паритет сред. Использование Docker и Docker Compose позволяет приблизить локальную среду разработки к production-среде. Это снижает вероятность ошибок, связанных с различиями окружений.

Логи и диагностика. В CI/CD-процессе при ошибке сборки или запуска контейнеров выводятся логи Docker Compose. Также для диагностики доступности backend используется healthcheck-маршрут `/api/health`.

Таким образом, выбранная архитектура соответствует требованиям курсовой работы и позволяет реализовать приложение как полноценную клиент-серверную систему с разделением ответственности между компонентами, поддержкой авторизации, ролевой модели, базы данных, административной панели, аналитики, Swagger-документации, контейнерного запуска, CI/CD-проверки и развёртывания на хостинге Render.

4. Выбор программного стека

Для реализации клиент-серверного приложения «Интерактивный конструктор образовательных тестов» был выбран современный стек технологий, обеспечивающий разработку полноценного fullstack-приложения.

4.1. Frontend: React, React Router, Redux Toolkit, Axios, CSS Modules

Клиентская часть приложения реализована с использованием библиотеки React. React позволяет создавать интерфейс на основе компонентного подхода, при котором каждая часть страницы оформляется как отдельный переиспользуемый компонент. Такой подход удобен для разработки приложения, в котором есть разные разделы: главная страница, регистрация, авторизация, список тестов, прохождение теста, конструктор вопросов, аналитика преподавателя и административная панель.

Для организации переходов между страницами используется React Router. Он обеспечивает маршрутизацию внутри одностраничного приложения без полной перезагрузки браузера. В проекте маршруты используются для перехода к списку тестов, конкретному тесту, профилю пользователя, конструктору, аналитике и панели администратора.

Управление состоянием клиентской части выполняется с помощью Redux Toolkit. В проекте состояние разделено на логические части. `authSlice` отвечает за хранение токена, данных пользователя, роли и состояния авторизации. `quizSlice` отвечает за вопросы теста, текущий вопрос, ответы пользователя, отправку результата и режим повтора ошибок. Такой подход упрощает обмен данными между компонентами и позволяет централизованно управлять логикой приложения.

Для взаимодействия с серверной частью используется отдельный API-сервис. В проекте применяется `Axios`, через который клиентская часть отправляет HTTP-запросы к backend. В файле `services/api.js` создаётся экземпляр `Axios` с базовым адресом из переменной окружения `REACT_APP_API_URL`; также настроен `interceptor`, который автоматически добавляет JWT-токен из `localStorage` в заголовок `Authorization` при обращении

к защищённым маршрутам.

Для стилизации интерфейса используются CSS Modules. Такой подход позволяет хранить стили рядом с компонентами и изолировать имена классов, что снижает вероятность конфликтов между разными частями интерфейса. CSS Modules применяются для страниц, компонентов теста, шапки сайта, конструктора, аналитики и административной панели.

В клиентской части также используются библиотеки для формирования отчётности. Для экспорта аналитики преподавателя в Excel применяется XLSX, что позволяет формировать табличный файл с результатами студентов. В коде страницы TeacherAnalytics импортируется библиотека xlsx, а данные аналитики группируются по студентам или по тестам.

4.2. Backend: Node.js, Express.js, Sequelize, JWT, bcryptjs, Swagger

Серверная часть приложения реализована на платформе Node.js. Использование JavaScript как на клиенте, так и на сервере упрощает разработку fullstack-приложения, так как единый язык применяется для реализации интерфейса, серверной логики и обработки данных.

В качестве серверного фреймворка используется Express.js. Он предоставляет удобные средства для создания REST API, подключения маршрутов, обработки HTTP-запросов и использования middleware. В backend-приложении Express используется для маршрутов авторизации, тестов, вопросов, результатов, аналитики и административной панели.

Серверная часть выполняет следующие функции:

- обработка запросов от клиентского приложения;
- регистрация и авторизация пользователей;
- формирование и проверка JWT-токена;
- проверка ролей пользователей;
- валидация входных данных;
- выполнение CRUD-операций для тестов и вопросов;
- сохранение результатов прохождения;

- расчёт правильных и неправильных ответов;
- выдача данных для аналитики преподавателя;
- обработка административных действий;
- управление системными настройками;
- предоставление Swagger-документации;
- предоставление healthcheck-маршрута.

Для работы с базой данных используется Sequelize. Это ORM-библиотека, позволяющая описывать таблицы базы данных в виде моделей JavaScript. В проекте через Sequelize описаны модели пользователей, тестов, вопросов, результатов прохождения и системных настроек. Backend подключается к PostgreSQL через строку DATABASE_URL, а при запуске синхронизирует модели с базой данных.

Для авторизации применяется JWT. После успешного входа backend формирует токен, который клиентская часть сохраняет и передаёт в заголовке Authorization. На защищённых маршрутах middleware проверяет токен и определяет пользователя. Проверка ролей используется для ограничения доступа к функциям преподавателя и администратора. В маршрутах также используется валидация через express-validator, например при регистрации, входе, сохранении результата и работе с вопросами.

Для безопасного хранения паролей используется bcryptjs. Пароль пользователя не сохраняется в базе данных в открытом виде: перед сохранением он хешируется, а при входе сравнивается с сохранённым хешем. При преобразовании пользователя в JSON пароль исключается из ответа сервера.

Для документирования API используется Swagger/OpenAPI. В серверном приложении подключается модуль Swagger, который формирует документацию доступных API-маршрутов. Это упрощает проверку backend-функциональности и описание интерфейса взаимодействия между клиентом и сервером. Также в backend реализован маршрут /api/health, возвращающий статус сервера, время и текущую среду запуска.

4.3. База данных: PostgreSQL и Sequelize

В качестве системы управления базами данных выбрана PostgreSQL. Она обеспечивает надёжное хранение данных, поддержку связей между таблицами и подходит для систем, где необходимо хранить пользователей, тесты, вопросы, результаты и настройки приложения.

В базе данных хранятся следующие основные сущности: пользователи, тесты, вопросы, результаты прохождения, системные настройки.

Для связи backend с PostgreSQL применяется Sequelize. ORM позволяет описывать модели, связи между ними и выполнять операции с базой данных из серверного кода. Это упрощает сопровождение проекта, так как структура данных описана на уровне приложения, а не только в SQL-скриптах.

PostgreSQL выбран по следующим причинам:

- поддерживает реляционную модель данных;
- обеспечивает целостность связей между сущностями;
- подходит для хранения структурированных данных образовательного приложения;
- может запускаться локально в Docker-контейнере;
- может использоваться как облачная база данных при размещении проекта на хостинге.

4.4. Инфраструктура: Docker, Docker Compose, Nginx

Для контейнеризации приложения используется Docker. Он позволяет упаковать отдельные части системы в контейнеры и запускать их независимо от особенностей локальной среды. В проекте контейнеризованы frontend, backend, база данных PostgreSQL и Nginx.

Для управления несколькими контейнерами применяется Docker Compose. Он позволяет запускать все компоненты системы одной командой и задавать связи между сервисами. В проекте используются сервисы frontend, backend, postgres и nginx, объединённые общей Docker-сетью. В локальной конфигурации открываются порты для frontend, backend, PostgreSQL и Nginx,

Для обработки внешних HTTP-запросов используется Nginx. Он выполняет роль веб-сервера и reverse proxy. В Docker-среде Nginx перенаправляет обычные запросы к frontend-сервису, а запросы с префиксом /api/ - к backend-сервису. В конфигурации также добавлены базовые заголовки безопасности, включая X-Frame-Options, X-Content-Type-Options, X-XSS-Protection и Strict-Transport-Security.

Для клиентской части используется многоэтапный Dockerfile. На первом этапе выполняется сборка React-приложения с передачей аргумента REACT_APP_API_URL, а на втором этапе собранные статические файлы размещаются в Nginx-контейнере. Также для frontend-контейнера задан healthcheck, который проверяет доступность приложения на порту 80.

4.5. Тестирование: Jest, React Testing Library, Supertest

Для проверки корректности приложения используются инструменты автоматизированного тестирования. В клиентской части применяются Jest и React Testing Library. Они позволяют проверять компоненты React, работу форм, отображение элементов интерфейса, обработку действий пользователя и состояние Redux-хранилища.

В проекте тестируются ключевые интерфейсные компоненты и страницы: шапка сайта, вопросы теста, элементы управления прохождением, регистрация, административная панель, аналитика и Redux-срезы состояния. Например, в тестах проверяется, что преподаватель видит пункты «Конструктор» и «Аналитика», а администратор видит пункт «Администрирование».

В серверной части используется Jest для проверки middleware, ролей, логики оценки ответов, системных настроек и других backend-модулей. Также для API-тестирования может использоваться Supertest, так как он подходит для проверки Express-приложений через HTTP-запросы без запуска отдельного внешнего сервера.

Автоматизированное тестирование выбрано потому, что проект содержит ролевую модель, защищённые маршруты, проверку ответов,

административные действия и работу с результатами. Для таких функций важно быстро выявлять ошибки после изменения кода.

4.6. CI/CD: GitHub Actions и Docker Hub

Для автоматизации проверки проекта используется GitHub Actions. В репозитории настроен workflow, который запускается при push, pull_request в ветки main и master, а также вручную через workflow_dispatch.

CI/CD-процесс состоит из нескольких этапов:

1. Получение кода из репозитория;
2. Установка Node.js;
3. Установка зависимостей клиентской и серверной частей через npm ci;
4. Запуск тестов;
5. Сборка и запуск Docker Compose stack;
6. Проверка backend через /api/health;
7. Проверка frontend;
8. Проверка Nginx-прокси через /api/health;
9. Вывод логов при ошибке;
10. Остановка контейнеров после проверки.

После успешной проверки контейнеров workflow также может публиковать Docker-образы в Docker Hub. Для этого используются docker/setup-buildx-action, docker/login-action и docker/build-push-action. Backend и frontend публикуются как отдельные образы с тегом latest и тегом, соответствующим SHA коммита.

Использование CI/CD позволяет автоматически проверять работоспособность проекта после изменений и снижает риск попадания нерабочей версии в репозиторий или production-среду.

4.7. Хостинг: Render

Для размещения приложения используется Render. Такой подход соответствует клиент-серверной архитектуре: frontend отвечает за пользовательский интерфейс, а backend предоставляет REST API и работает с

базой данных.

Frontend-сервис использует переменную окружения `REACT_APP_API_URL`, в которой указывается адрес backend-сервиса. Backend-сервис использует переменные окружения `PORT`, `NODE_ENV`, `DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRE`, `FRONTEND_URL` и `CORS_ORIGINS`.

Render выбран как удобный вариант для размещения учебного fullstack-приложения, так как он позволяет разделить frontend и backend, подключить облачную базу данных, использовать переменные окружения и получить публичные URL для проверки работы приложения.

4.8. Система контроля версий: Git

Для управления исходным кодом используется Git. Он позволяет отслеживать изменения, хранить историю разработки, возвращаться к предыдущим версиям и использовать удалённый репозиторий для хранения проекта.

В репозитории проект организован с разделением на клиентскую и серверную части, конфигурационные файлы Docker, Nginx, GitHub Actions и файлы переменных окружения-примеров. Такое разделение делает проект удобным для сопровождения и развёртывания.

Git также необходим для работы CI/CD, так как GitHub Actions запускает проверки на основе событий в репозитории: отправки изменений, создания pull request или ручного запуска workflow.

4.9. Обоснование выбора технологий

Выбранный стек технологий соответствует требованиям к разработке современного fullstack-приложения.

React обеспечивает создание динамического пользовательского интерфейса и удобное разделение интерфейса на компоненты. React Router используется для маршрутизации между страницами без перезагрузки браузера. Redux Toolkit упрощает централизованное хранение состояния

пользователя, тестов и прохождения тестирования. Axios позволяет вынести работу с API в отдельный сервис и автоматически добавлять JWT-токен к защищённым запросам.

Node.js и Express.js позволяют реализовать REST API, middleware, авторизацию, проверку ролей, обработку ошибок и бизнес-логику приложения. Sequelize упрощает работу с PostgreSQL и позволяет описывать структуру данных через модели. PostgreSQL обеспечивает надёжное хранение пользователей, тестов, вопросов, результатов и системных настроек.

Docker и Docker Compose делают приложение переносимым и позволяют запускать все компоненты системы в изолированных контейнерах. Nginx выполняет роль reverse proxy и веб-сервера для frontend. GitHub Actions автоматизирует проверку кода, запуск тестов и проверку контейнеров. Docker Hub используется для публикации образов, а Render - для размещения frontend и backend в production-среде.

Таблица 7 - Используемый программный стек

Категория	Технология	Назначение
Frontend	React	Построение пользовательского интерфейса
Frontend	React Router	Маршрутизация между страницами без перезагрузки браузера
Frontend	Redux Toolkit	Управление состоянием авторизации и прохождения тестов
Frontend	Axios	Отправка HTTP-запросов к backend и добавление JWT-токена
Frontend	CSS Modules	Изолированная стилизация компонентов и страниц
Frontend	XLSX	Экспорт аналитики преподавателя в Excel
Backend	Node.js	Среда выполнения серверной части
Backend	Express.js	Создание REST API и обработка HTTP-запросов
Backend	Sequelize	ORM для работы с PostgreSQL
Backend	express-validator	Валидация входных данных
Backend	jsonwebtoken	Создание и проверка JWT-токенов
Backend	bcryptjs	Хеширование и проверка паролей
Backend	CORS	Настройка доступа frontend к backend
Backend	dotenv	Загрузка переменных окружения
Backend	Swagger/OpenAPI	Документирование REST API
База данных	PostgreSQL	Хранение пользователей, тестов, вопросов, результатов и настроек
Тестирование	Jest	Модульное тестирование frontend и backend
Тестирование	React Testing	Тестирование React-компонентов

	Library	
Тестирование	Supertest	Проверка API Express-приложения
Инфраструктура	Docker	Контейнеризация frontend, backend и базы данных
Инфраструктура	Docker Compose	Запуск нескольких сервисов одной конфигурацией
Инфраструктура	Nginx	Reverse проху и отдача статических файлов frontend
CI/CD	GitHub Actions	Автоматическая установка зависимостей, запуск тестов и проверка контейнеров
CI/CD	Docker Hub	Публикация Docker-образов frontend и backend
Хостинг	Render	Размещение frontend, backend и подключение production-среды
Контроль версий	Git	Управление исходным кодом и историей изменений

5. Проектирование базы данных

База данных приложения «Интерактивный конструктор образовательных тестов» предназначена для хранения пользователей, тестов, вопросов, результатов прохождения и системных настроек. В проекте используется PostgreSQL, а взаимодействие серверной части с базой данных выполняется через Sequelize.

Подключение к базе данных задаётся через переменную окружения DATABASE_URL. При запуске backend синхронизирует модели Sequelize с базой данных.

5.1. Основные сущности

В рамках проектирования базы данных выделены следующие основные сущности.

Таблица 8 - Сущности БД

Таблица	Назначение
users	Хранение пользователей системы
tests	Хранение образовательных тестов
questions	Хранение вопросов, вариантов ответов и правильных ответов
test_results	Хранение результатов прохождения тестов

Сущность users хранит данные пользователей. Для каждого пользователя сохраняются идентификатор, имя, email, пароль в хешированном виде, роль и дата создания.

Таблица 9 - Основные поля users

Поле	Назначение
id	Идентификатор пользователя
email	Email пользователя
password	Хеш пароля
name	Имя пользователя
role	Роль: student, teacher, admin
createdAt	Дата создания пользователя

Сущность tests хранит данные тестов. Каждый тест связан с автором через поле authorId.

Таблица 10 - Основные поля tests

Поле	Назначение
id	Идентификатор теста
title	Название теста
description	Описание теста
questionLimit	Ограничение количества вопросов
authorId	Автор теста
isActive	Активность теста
createdAt	Дата создания теста

Сущность questions хранит вопросы конкретного теста. Один вопрос относится к одному тесту через поле testId.

Таблица 11 - Основные поля questions

Поле	Назначение
id	Идентификатор вопроса
testId	Идентификатор теста
type	Тип вопроса: single, multiple, matching
questionText	Текст вопроса
options	Варианты ответов для single и multiple
left	Левая часть для вопроса на сопоставление
right	Правая часть для вопроса на сопоставление
correct	Правильный ответ или структура правильных ответов
order	Порядок вопроса в тесте

Сущность test_results хранит результаты прохождения тестов. Ответы пользователя не вынесены в отдельную таблицу, а сохраняются в JSON-поле answers.

Таблица 12 - Основные поля test_results

Поле	Назначение
id	Идентификатор результата
userId	Пользователь, прошедший тест
testId	Пройденный тест
score	Количество правильных ответов
totalQuestions	Общее количество вопросов
durationSeconds	Время прохождения в секундах
answers	Ответы пользователя в формате JSON
completedAt	Дата завершения теста

5.2. Связи между сущностями

Между сущностями базы данных устанавливаются следующие связи.

Таблица 13 - Связи между сущностями

Связь	Тип связи	Описание
users -> tests	один ко многим	Один пользователь может создать несколько

		тестов
tests -> questions	один ко многим	Один тест содержит несколько вопросов
users -> test_results	один ко многим	Один пользователь может иметь несколько результатов
tests -> test_results	один ко многим	Один тест может быть пройден разными пользователями

Связь users -> tests используется для определения автора теста. Это необходимо для проверки прав преподавателя: преподаватель может управлять своими тестами, а администратор - всеми тестами.

Связь tests -> questions используется для конструктора тестов. Сначала создаётся тест, затем к нему добавляются вопросы.

Связь users -> test_results позволяет хранить историю прохождения тестов конкретным пользователем.

Связь tests -> test_results позволяет собирать аналитику по конкретному тесту.

5.3. Хранение вопросов разных типов

В приложении поддерживаются три типа вопросов.

Таблица 14 - Особенности хранения вопросов

Тип вопроса	Назначение	Хранение данных
single	Один правильный ответ	options хранит варианты, correct хранит правильный вариант
multiple	Несколько правильных ответов	options хранит варианты, correct хранит массив правильных вариантов
matching	Сопоставление элементов	left и right хранят элементы пар, correct хранит соответствия

Для вопросов типа single пользователь выбирает один вариант ответа.

Для вопросов типа multiple пользователь выбирает несколько вариантов ответа.

Для вопросов типа matching используются два массива: left и right. В поле correct сохраняется структура правильных соответствий.

Все поля, связанные с вариантами и правильными ответами, хранятся в формате JSON. Это позволяет использовать одну таблицу questions для разных типов вопросов.

5.4. Хранение результатов прохождения

После завершения теста backend рассчитывает результат и создаёт запись в таблице test_results.

Таблица 15 – Новые записи в таблице test_results

Данные	Поле	Данные
Пользователь	userId	Пользователь
Тест	testId	Тест
Количество правильных ответов	score	Количество правильных ответов
Общее количество вопросов	totalQuestions	Общее количество вопросов
Время прохождения	durationSeconds	Время прохождения
Ответы пользователя	answers	Ответы пользователя
Дата завершения	completedAt	Дата завершения

Поле answers используется для хранения подробных ответов пользователя. Благодаря этому можно реализовать повтор ошибок и аналитику без отдельной таблицы ответов.

5.5. Итоговая структура базы данных

Итоговая структура базы данных включает пять основных таблиц:

Таблица 16 – Структура БД

Таблица	Ключевая роль
users	Пользователи и роли
tests	Тесты, авторы, лимиты и активность
questions	Вопросы разных типов
test_results	Результаты, ответы и время прохождения

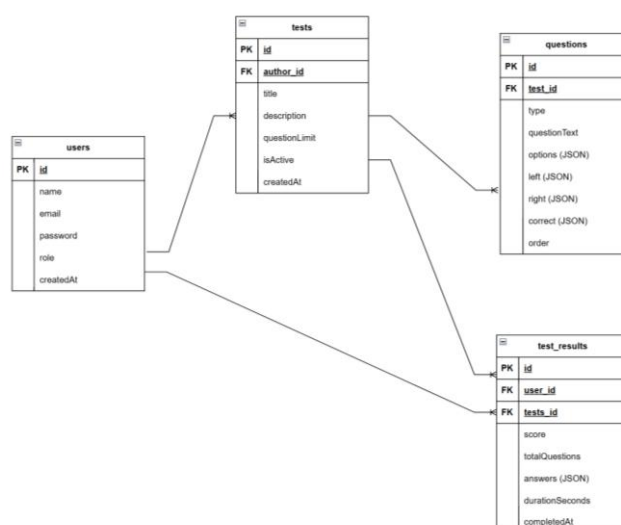


Рисунок 6 - ER-диаграмма базы данных приложения «Интерактивный конструктор образовательных тестов»

6. Разработка клиентской части

Клиентская часть приложения «Интерактивный конструктор образовательных тестов» реализована как одностраничное React-приложение. Она отвечает за всё, что видит пользователь в браузере: страницы входа и регистрации, список тестов, прохождение теста, конструктор вопросов, аналитику преподавателя и административную панель.

Точка входа приложения находится в `client/src/index.js`, где React-приложение подключается к DOM-элементу `root`. Основная структура клиентской части задаётся в `App.js`: в нём подключается Redux-хранилище, настраивается маршрутизация и выполняется проверка авторизации пользователя при запуске приложения.

6.1. Структура React-приложения

Код клиентской части расположен в директории `client`. Основная логика находится в каталоге `client/src`. Внутри него проект разделён на страницы, компоненты, Redux-хранилище, API-сервис и файлы стилей.

Страницы приложения расположены в каталоге `pages`. Здесь находятся главная страница, страницы входа и регистрации, профиль пользователя, список тестов, административная панель, конструктор вопросов и страница аналитики преподавателя.

Переиспользуемые элементы интерфейса находятся в каталоге `components`. Например, компонент `Header` отвечает за верхнюю навигацию, компоненты `QuizContainer`, `Question`, `QuizControls` и `Score` используются при прохождении теста.

Такое разделение делает клиентскую часть более понятной: страницы отвечают за крупные сценарии пользователя, а компоненты - за отдельные элементы интерфейса.

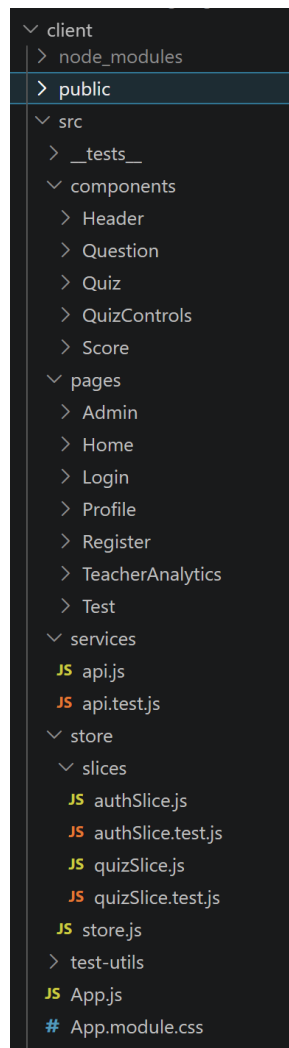


Рисунок 7 - Структура клиентской части приложения

6.2. Маршрутизация страниц

Маршрутизация реализована с помощью `react-router-dom`. В `App.js` используется `BrowserRouter`, внутри которого описаны основные маршруты приложения. Всё приложение также обернуто в `Provider`, чтобы компоненты могли получать данные из `Redux`-хранилища.

В приложении используются маршруты для главной страницы, списка тестов, прохождения конкретного теста, входа, регистрации, профиля, аналитики, административной панели и конструктора вопросов. Маршрут `/test/:testId` открывает прохождение конкретного теста по его идентификатору. Маршруты `/constructor` и `/admin/questions` ведут к интерфейсу управления тестами и вопросами. Маршрут `/analytics` используется для аналитики преподавателя, а `/admin` - для панели администратора.

При загрузке приложения выполняется проверка авторизации. Если в хранилище есть JWT-токен, но данные пользователя ещё не загружены, вызывается действие `fetchProfile`. Благодаря этому пользователь остаётся авторизованным даже после обновления страницы.

```
<Router>
  <div className={styles.app}>
    <Header />
    <main className={styles.main}>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/tests" element={<Test />} />
        <Route path="/test/:testId" element={<QuizContainer />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/profile" element={<Profile />} />
        <Route path="/analytics" element={<TeacherAnalytics />} />
        <Route path="/admin" element={<AdminDashboard />} />
        <Route path="/admin/questions" element={<QuestionsAdmin />} />
        <Route path="/constructor" element={<QuestionsAdmin />} />
      </Routes>
    </main>
  </div>
</Router>
```

Рисунок 8 - Основные маршруты приложения

6.3. Навигация и разделение интерфейса по ролям

Навигация реализована в компоненте `Header`. Он отображает ссылки на основные разделы приложения и меняет набор доступных пунктов в зависимости от роли пользователя.

Неавторизованный пользователь видит ссылки на главную страницу, список тестов, вход и регистрацию. После входа в систему пользователь получает доступ к профилю и кнопке выхода.

Студент может просматривать тесты, проходить их и открывать профиль. Преподаватель дополнительно видит ссылки на конструктор и аналитику. Администратор получает доступ к конструктору и административной панели.

6.4. Главная страница

Главная страница является стартовой страницей приложения. Она кратко объясняет назначение системы и направляет пользователя к основным действиям: просмотру тестов, регистрации, входу в систему или работе с доступными разделами после авторизации.

Основная задача главной страницы - быстро показать, что приложение предназначено для создания и прохождения образовательных тестов. Поэтому она выполняет информационную и навигационную функцию.

6.5. Страница тестов

Страница тестов отображает список доступных образовательных тестов. Данные загружаются с backend через API. Каждый тест выводится в виде карточки с названием, описанием, количеством вопросов, кнопкой перехода к прохождению, также показывает ограничение по времени, рассчитанное на основе количества вопросов. Если список тестов пуст, пользователь видит сообщение об отсутствии доступных тестов. Это нужно для корректной обработки ситуации, когда тесты ещё не созданы или временно недоступны.

Переход к прохождению выполняется по маршруту `/test/:testId`, где `testId` - идентификатор выбранного теста.

6.6. Прохождение теста

Прохождение теста реализовано в компоненте `QuizContainer`. Он получает `testId` из адресной строки, загружает вопросы выбранного теста, хранит текущий прогресс и управляет отправкой результата на сервер.

Внутри `QuizContainer` используются отдельные компоненты. `Question` отображает текст вопроса и варианты ответа. `QuizControls` отвечает за переход между вопросами. `Score` показывает прогресс прохождения и итоговый результат.

Приложение поддерживает три типа вопросов: с одним правильным ответом, с несколькими правильными ответами и на сопоставление. Для вопроса с одним ответом пользователь выбирает один вариант. Для вопроса с

несколькими ответами можно отметить несколько вариантов. Для вопроса на сопоставление пользователь выбирает соответствия между элементами.

После завершения теста ответы отправляются на backend. Сервер рассчитывает количество правильных ответов, сохраняет результат и возвращает данные для отображения пользователю. На экране результата показывается количество правильных ответов, процент выполнения, текстовая оценка результата и кнопка повторного прохождения.

Также реализован режим повтора ошибок. В этом режиме пользователь проходит только те вопросы, на которые ранее ответил неправильно. Это делает приложение полезным не только для контроля знаний, но и для повторения материала.

6.7. Конструктор тестов и вопросов

Конструктор реализован в компоненте QuestionsAdmin. Он доступен преподавателю и администратору. Если пользователь не имеет нужной роли, интерфейс не даёт ему работать с этим разделом.

В конструкторе можно создавать новые тесты, выбирать уже существующие, редактировать название и описание, задавать лимит вопросов, добавлять вопросы, изменять их и удалять. Также доступно удаление выбранного теста.

При создании вопроса пользователь выбирает его тип: single, multiple или matching. Для каждого типа интерфейс отображает подходящую форму. Для обычных вопросов задаются варианты ответа и правильный вариант, а для вопросов на сопоставление - пары элементов и правильные соответствия.

Перед отправкой данных на сервер клиентская часть подготавливает объект вопроса. Пустые варианты отбрасываются, проверяется наличие правильного ответа, а для вопросов на сопоставление проверяется заполнение всех пар.

6.8. Административная панель

Административная панель реализована в компоненте AdminDashboard и

доступна только пользователю с ролью администратора.

В этом разделе администратор управляет пользователями, тестами и системными настройками. Он может просматривать список пользователей, изменять их роли и удалять учётные записи. Также администратор может просматривать тесты и включать или отключать их активность.

Отдельная часть административной панели отвечает за настройки приложения. Через неё можно изменить название платформы, включить или отключить самостоятельную регистрацию пользователей, а также настроить необходимость модерации тестов преподавателей.

Административная панель вынесена в отдельную страницу, потому что она относится не к учебному процессу, а к управлению всей системой.

6.9. Аналитика преподавателя

Страница аналитики реализована в компоненте TeacherAnalytics и доступна пользователю с ролью преподавателя.

На этой странице преподаватель видит результаты прохождения тестов студентами. Интерфейс показывает общее количество прохождений, количество студентов, количество тестов и средний результат. Данные можно просматривать в разрезе студентов или тестов.

Аналитика позволяет преподавателю понять, какие тесты вызывают больше трудностей, какие студенты показывают лучшие или худшие результаты, а также какие ответы были правильными и неправильными.

Преподаватель может выгрузить результаты в PDF или Excel. Это удобно для подготовки отчётности или дальнейшего анализа данных вне приложения.

6.10. Работа с состоянием через Redux

Для управления состоянием используется Redux Toolkit. Это позволяет хранить важные данные приложения централизованно и не передавать их вручную между большим количеством компонентов.

Состояние авторизации хранится в authSlice. В нём находятся JWT-

токен, данные пользователя, статус загрузки, ошибка и признак авторизации. Через этот slice выполняются регистрация, вход, загрузка профиля и выход из системы.

Состояние прохождения теста хранится в `quizSlice`. В нём находятся список вопросов, текущий вопрос, ответы пользователя, результат прохождения, ошибочные вопросы и режим повтора ошибок.

Такое разделение удобно, потому что авторизация и прохождение теста являются разными частями логики приложения. Изменения в одной части не мешают другой, а компоненты получают только те данные, которые им нужны.

6.11. Взаимодействие с backend

Для связи с серверной частью используется файл `services/api.js`. В нём создаётся общий экземпляр `Axios`, через который выполняются все запросы к backend.

Базовый адрес сервера берётся из переменной окружения `REACT_APP_API_URL`. Это позволяет использовать один адрес API при локальной разработке и другой адрес при размещении приложения на хостинге.

В API-сервисе настроен `interceptor`, который автоматически добавляет JWT-токен из `localStorage` в заголовок `Authorization`. Благодаря этому защищённые запросы к серверу выполняются без повторения одной и той же логики в каждом компоненте.

Запросы разделены по смыслу: авторизация, тесты, вопросы, результаты, административные функции, системные настройки и аналитика преподавателя. Такой подход делает работу с backend более организованной и упрощает дальнейшее расширение клиентской части.

7. Разработка серверной части

Серверная часть приложения «Интерактивный конструктор образовательных тестов» реализована на Node.js с использованием Express.js. Она отвечает за обработку HTTP-запросов от клиентского приложения, выполнение бизнес-логики, авторизацию пользователей, проверку ролей, валидацию входных данных, расчёт результатов тестирования и взаимодействие с базой данных PostgreSQL.

Backend запускается как отдельный сервис и получает настройки через переменные окружения. Для подключения к базе данных используется DATABASE_URL, для подписи токенов - JWT_SECRET, а срок действия токена задаётся через JWT_EXPIRE.

7.1. Структура backend-приложения

Серверная часть расположена в директории server. Основная логика находится в каталоге server/src, где выделены маршруты, контроллеры, модели базы данных, middleware, конфигурация Swagger, константы ролей и служебные модули.

Маршруты принимают HTTP-запросы и передают их в контроллеры. Контроллеры выполняют бизнес-логику: создают пользователей, проверяют пароли, работают с тестами, сохраняют результаты, формируют аналитику и выполняют административные действия. Middleware используются для проверки JWT-токена, ролей пользователя и корректности входных данных. Модели Sequelize описывают таблицы базы данных и связи между ними.

В новой версии backend дополнен административным модулем, модулем системных настроек, маршрутом аналитики преподавателя, Swagger-документацией и healthcheck-маршрутом.

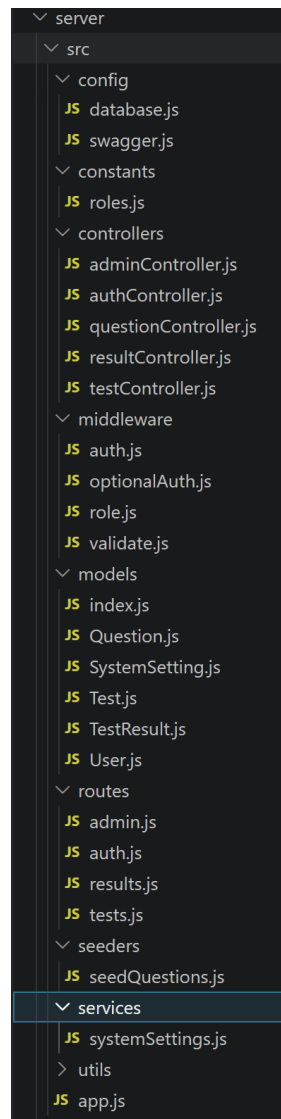


Рисунок 9 - Структура backend-приложения

7.2. REST API

Серверная часть предоставляет REST API, через который frontend выполняет регистрацию, вход, получение профиля, загрузку тестов, управление вопросами, прохождение тестов, получение результатов, аналитику и административные операции.

API разделён на несколько групп:

- /api/auth - регистрация, вход и получение профиля пользователя;
- /api/tests - получение тестов, создание, редактирование, удаление тестов, работа с вопросами и отправка ответов;
- /api/results - сохранение результатов, получение истории прохождений, повтор ошибок, статистика теста и аналитика преподавателя;

- /api/admin - управление пользователями, ролями, активностью тестов и системными настройками;
- /api/health - проверка работоспособности backend;
- /api/docs и /api/docs.json - Swagger-документация API.

В клиентском API-сервисе используются запросы к /api/auth/register, /api/auth/login, /api/auth/profile, маршрутам тестов, результатов и административным маршрутам /api/admin/users, /api/admin/tests, /api/admin/settings.

7.3. Маршруты авторизации

Маршруты авторизации находятся в группе /api/auth.

Регистрация выполняется через POST /api/auth/register. При регистрации сервер проверяет формат email, минимальную длину пароля, длину имени и отсутствие HTML-тегов в имени. Если в запросе передаётся роль, она может быть только student, то есть публичная регистрация не позволяет создать преподавателя или администратора. Это защищает систему от самостоятельного получения расширенных прав.

Вход выполняется через POST /api/auth/login. Пользователь передаёт email и пароль, после чего сервер ищет пользователя в базе данных, сравнивает пароль с хешем и при успешной проверке возвращает JWT-токен.

Профиль пользователя получается через GET /api/auth/profile. Этот маршрут защищён middleware авторизации и доступен только пользователю с корректным токеном.

7.4. Работа с тестами и вопросами

Маршруты тестов находятся в группе /api/tests. Они используются как для просмотра тестов студентами, так и для управления тестами преподавателями и администраторами.

Для тестов реализованы операции получения списка, получения теста по id, создания, редактирования и удаления. Также есть маршрут /api/tests/manage, который используется для получения тестов, доступных

пользователю для управления.

Для вопросов используются вложенные маршруты внутри теста. Обычный маршрут `/api/tests/:id/questions` возвращает вопросы для прохождения. Маршрут `/api/tests/:id/questions/manage` используется в конструкторе и доступен только пользователям с правами управления.

Создание, редактирование и удаление вопросов доступны преподавателю и администратору. При этом backend проверяет не только роль, но и корректность данных вопроса. Для вопроса проверяется текст, тип, правильный ответ, варианты ответа, а для matching-вопросов - заполненность левой и правой частей. Валидация маршрутов тестов и вопросов реализована через express-validator.

В новой версии у теста также учитываются поля `questionLimit` и `isActive`. `questionLimit` ограничивает количество вопросов, которые используются при прохождении теста, а `isActive` определяет, доступен ли тест обычным пользователям.

7.5. Отправка ответов и сохранение результатов

После прохождения теста frontend отправляет ответы на сервер. В проекте используются два связанных механизма: отправка ответов на `/api/tests/:id/submit` и сохранение результата через `/api/results`.

Результаты обрабатываются в группе `/api/results`. При сохранении результата сервер проверяет `testId`, а также может принять `durationSeconds` - время прохождения теста в секундах.

В результат сохраняются пользователь, тест, количество правильных ответов, общее количество вопросов, ответы пользователя, дата завершения и длительность прохождения. Ответы пользователя хранятся внутри результата в JSON-структуре.

Для пользователя доступен маршрут GET `/api/results/my`, который возвращает его историю прохождения. Для повтора ошибок используется GET `/api/results/:id/mistakes`, который получает вопросы, на которые пользователь ранее ответил неправильно.

7.6. Аналитика преподавателя

Backend реализует аналитику результатов для преподавателя. Для этого используется маршрут: GET /api/results/teacher/performance.

Он доступен только пользователю с ролью teacher. Сервер возвращает данные по прохождению тестов: идентификатор результата, название теста, студента, количество правильных и неправильных ответов, процент выполнения, длительность прохождения, дату завершения и детали ответов. Swagger-схема также описывает объект TeacherPerformanceResult и отчёт TeacherPerformanceReport.

Также для преподавателя и администратора доступна статистика конкретного теста: GET /api/results/test/:testId/stats. Этот маршрут защищён авторизацией и проверкой роли teacher или admin.

7.7. Административные маршруты

Административные маршруты находятся в группе /api/admin. Весь роутер защищён middleware auth и role('admin'), поэтому доступ к нему имеет только администратор.

Через административный API реализованы следующие действия:

- GET /api/admin/users - получение списка пользователей;
- PATCH /api/admin/users/:id/role - изменение роли пользователя;
- DELETE /api/admin/users/:id - удаление пользователя;
- GET /api/admin/tests - получение списка тестов для администрирования;
- PATCH /api/admin/tests/:id/moderation - включение или отключение активности теста;
- GET /api/admin/settings - получение системных настроек;
- PATCH /api/admin/settings - изменение системных настроек.

При изменении роли сервер проверяет, что новое значение входит в список допустимых ролей: student, teacher, admin.

7.8. Авторизация и проверка ролей

В приложении используется JWT-авторизация. После успешного входа

сервер формирует токен, который клиент передаёт в заголовке Authorization в формате Bearer <token>. Middleware авторизации проверяет токен, извлекает пользователя и передаёт его дальше в обработчик запроса.

В системе предусмотрены три роли: student, teacher и admin. Роль по умолчанию - student. Роли используются для ограничения доступа к функциям приложения.

7.9. Валидация входных данных

Для проверки входных данных используется express-validator. Валидация выполняется на маршрутах авторизации, тестов, вопросов, результатов и административных операций.

Проверяются email, пароль, имя пользователя, идентификаторы пользователей, тестов и вопросов, тип вопроса, варианты ответа, правильные ответы, роль пользователя, флаги активности и системные настройки.

После правил валидации вызывается общий middleware validate, который собирает ошибки и возвращает клиенту сообщение, если данные некорректны. Такой подход позволяет не допускать некорректные значения до контроллеров и базы данных.

7.10. Работа с PostgreSQL

Для работы с PostgreSQL используется Sequelize. В проекте определены модели User, Test, Question, TestResult.

Модель User хранит email, имя, хеш пароля и роль пользователя. Пароль хешируется через bcryptjs перед созданием или обновлением записи. В модели также есть метод comparePassword для проверки пароля и переопределённый toJSON, который удаляет пароль из ответа сервера.

Модель Test хранит название, описание, автора, лимит вопросов и признак активности. Модель Question хранит текст вопроса, тип, варианты, данные для сопоставления и правильные ответы. Модель TestResult хранит пользователя, тест, результат, общее количество вопросов, ответы, время прохождения и дату завершения.

Связи между моделями описывают основную логику приложения: пользователь создаёт тесты, тест содержит вопросы, пользователь имеет результаты прохождения, а результат связан с конкретным тестом.

7.11. Swagger-документация и healthcheck

Для документирования API используется Swagger/OpenAPI. В проекте описаны основные группы API, включая авторизацию, тесты, результаты и административные маршруты. В Swagger также описана bearer-авторизация через JWT. Документация доступна через /api/docs, а JSON-описание - через /api/docs.json. Наличие этих маршрутов проверяется в тестах Swagger-конфигурации.

Для проверки работоспособности backend используется маршрут /api/health. Он нужен для ручной диагностики и для автоматической проверки в CI/CD или Docker-среде.

7.12. Основные API-маршруты

Таблица 17 - Основные API-маршруты

Метод	Endpoint	Назначение	Доступ
POST	/api/auth/register	Регистрация пользователя	Публичный
POST	/api/auth/login	Вход пользователя	Публичный
GET	/api/auth/profile	Получение профиля	Авторизованный пользователь
GET	/api/tests	Получение списка активных тестов	Публичный
GET	/api/tests/manage	Получение тестов для управления	Преподаватель, администратор
GET	/api/tests/:id	Получение данных теста	Публичный / авторизованный
GET	/api/tests/:id/questions	Получение вопросов для прохождения	Публичный / авторизованный
POST	/api/tests/:id/submit	Отправка ответов и расчёт результата	Авторизованный пользователь
POST	/api/tests	Создание теста	Преподаватель, администратор
PUT	/api/tests/:id	Редактирование теста	Автор теста или администратор
DELETE	/api/tests/:id	Удаление теста	Автор теста или администратор
GET	/api/tests/:id/questions/manage	Вопросы для конструктора	Преподаватель, администратор

POST	/api/tests/:id/questions	Добавление вопроса	Преподаватель, администратор
PUT	/api/tests/:id/questions/:questionId	Редактирование вопроса	Преподаватель, администратор
DELETE	/api/tests/:id/questions/:questionId	Удаление вопроса	Преподаватель, администратор
POST	/api/results	Сохранение результата	Авторизованный пользователь
GET	/api/results/my	Получение своих результатов	Авторизованный пользователь
GET	/api/results/:id/mistakes	Получение ошибок результата	Авторизованный пользователь
GET	/api/results/test/:testId/stats	Статистика теста	Преподаватель, администратор
GET	/api/results/teacher/performance	Аналитика преподавателя	Преподаватель
GET	/api/admin/users	Список пользователей	Администратор
PATCH	/api/admin/users/:id/role	Изменение роли пользователя	Администратор
DELETE	/api/admin/users/:id	Удаление пользователя	Администратор
GET	/api/admin/tests	Список тестов для администрирования	Администратор
PATCH	/api/admin/tests/:id/moderation	Изменение активности теста	Администратор
GET	/api/admin/settings	Получение настроек системы	Администратор
PATCH	/api/admin/settings	Обновление настроек системы	Администратор
GET	/api/health	Проверка работоспособности backend	Публичный
GET	/api/docs	Swagger UI	Публичный
GET	/api/docs.json	OpenAPI JSON	Публичный

8. Тестирование приложения

Тестирование приложения «Интерактивный конструктор образовательных тестов» проводилось для проверки корректности основных пользовательских сценариев, клиентской и серверной логики, ролевой модели, административных функций, аналитики преподавателя, устойчивости API к некорректным данным и работоспособности контейнеризованного развёртывания.

8.1. Регистрация и авторизация

Первым этапом была проверена регистрация пользователя и последующая авторизация в системе. Этот сценарий является базовым, так как от него зависит доступ к профилю, прохождению тестов, сохранению результатов, конструктору, аналитике и административной панели.

При регистрации проверялось, что пользователь может создать учётную запись, а роль по умолчанию устанавливается как student. Также проверялось, что при публичной регистрации нельзя самостоятельно создать пользователя с ролью teacher или admin.

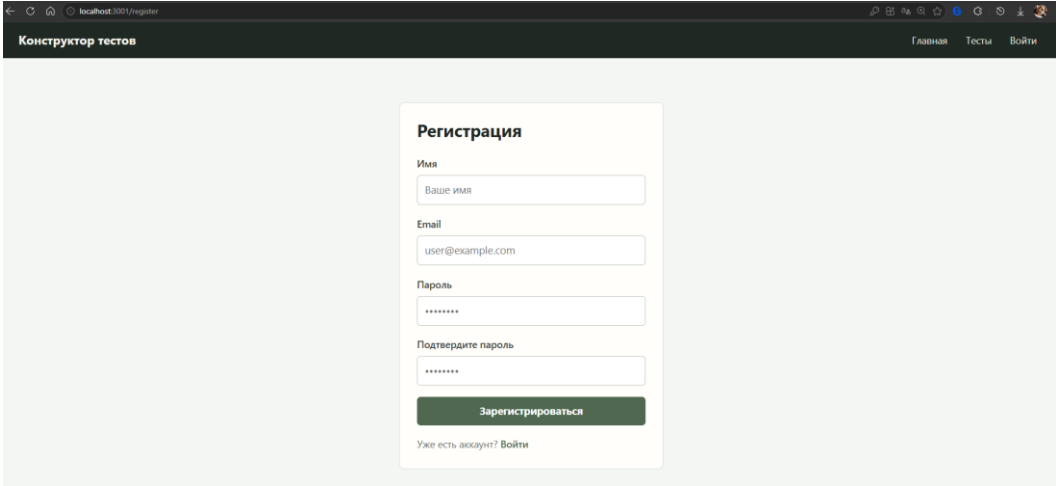
The image shows a web browser window with the address bar displaying 'localhost:3001/register'. The page title is 'Конструктор тестов'. In the top right corner, there are links for 'Главная', 'Тесты', and 'Войти'. The main content is a registration form titled 'Регистрация'. It contains four input fields: 'Имя' (with placeholder 'Ваше имя'), 'Email' (with placeholder 'user@example.com'), 'Пароль', and 'Подтвердите пароль' (both with placeholder '*****'). Below these fields is a green button labeled 'Зарегистрироваться'. At the bottom of the form, there is a link that says 'Уже есть аккаунт? Войти'.

Рисунок 10 - Форма регистрации пользователя

После заполнения формы регистрации система создаёт пользователя и отображает успешный результат.

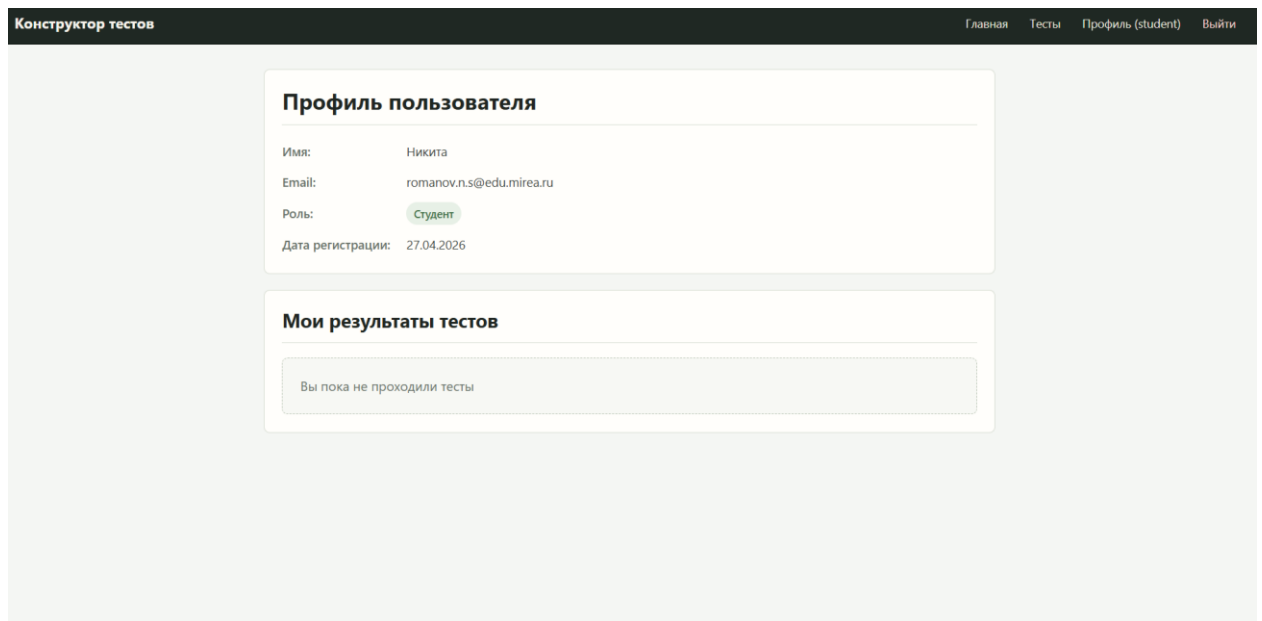


Рисунок 11 - Успешная регистрация пользователя

Далее выполнялась проверка авторизации. Пользователь вводит email и пароль, после чего backend проверяет данные, сравнивает пароль с хешем и возвращает JWT-токен. После успешного входа пользователь получает доступ к функциям приложения в соответствии со своей ролью.

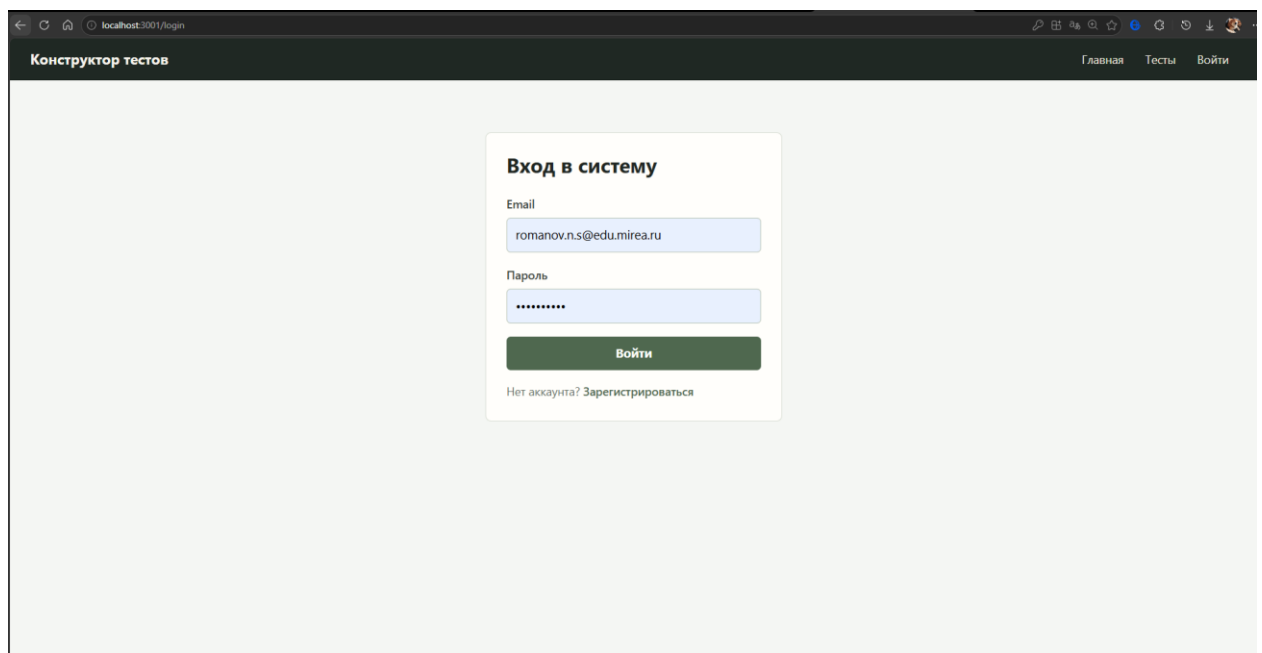


Рисунок 12 - Форма авторизации пользователя

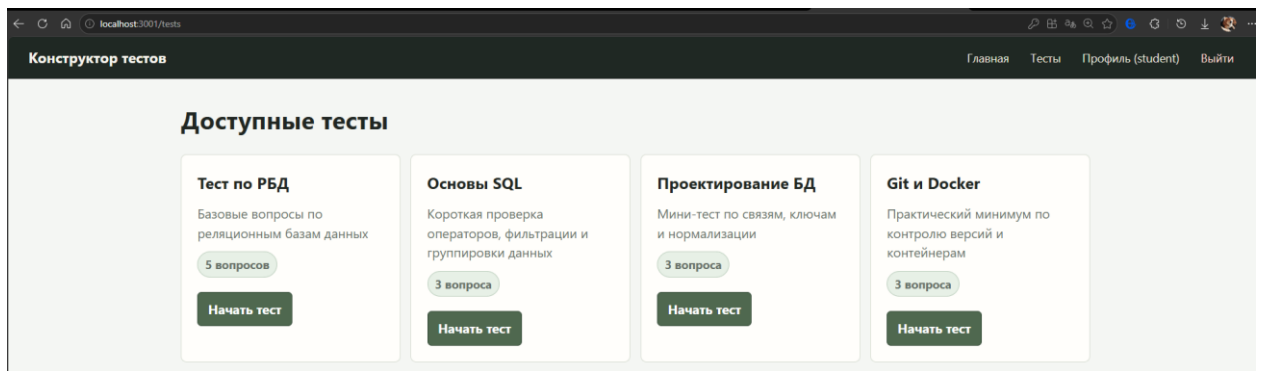


Рисунок 13 - Успешная авторизация пользователя

Также проверялось восстановление сессии после обновления страницы. Если токен сохранён, клиентская часть вызывает получение профиля пользователя и восстанавливает состояние авторизации.

8.2. Проверка основных пользовательских сценариев

После регистрации и входа были протестированы основные сценарии работы приложения.

8.2.1. Просмотр списка тестов

Пользователь открывает страницу со списком тестов. Клиентская часть отправляет запрос к backend и отображает доступные активные тесты. Для каждого теста выводится название, описание, количество вопросов и примерное время прохождения.

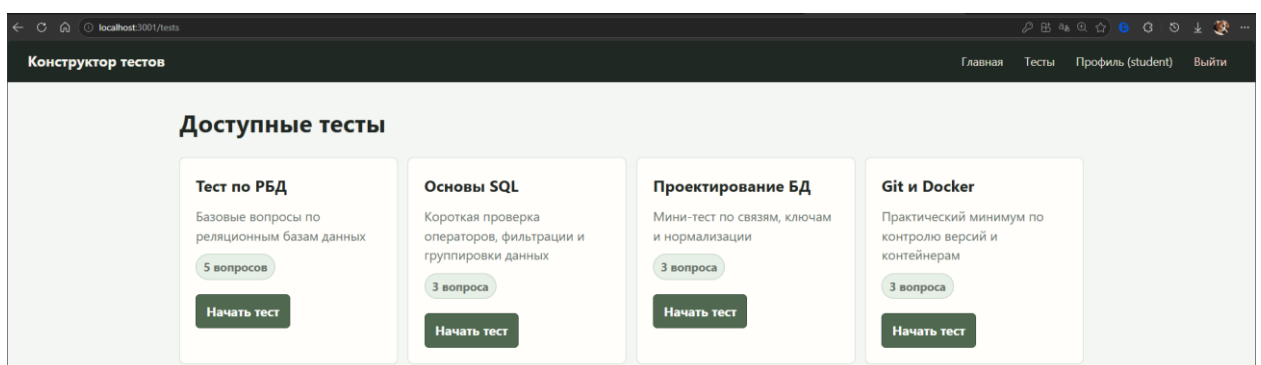


Рисунок 14 - Список доступных тестов

8.2.2. Прохождение теста

Пользователь выбирает тест и переходит на страницу прохождения. Компонент QuizContainer загружает вопросы, отображает текущий вопрос, сохраняет ответы пользователя и отправляет результат после завершения.

В ходе тестирования проверялись следующие действия:

- загрузка вопросов по testId;
- отображение текущего вопроса;
- выбор одного ответа для вопроса single;
- выбор нескольких ответов для вопроса multiple;
- сопоставление элементов для вопроса matching;
- переход между вопросами;
- отправка результата;
- отображение итогового количества правильных ответов;
- сохранение времени прохождения.

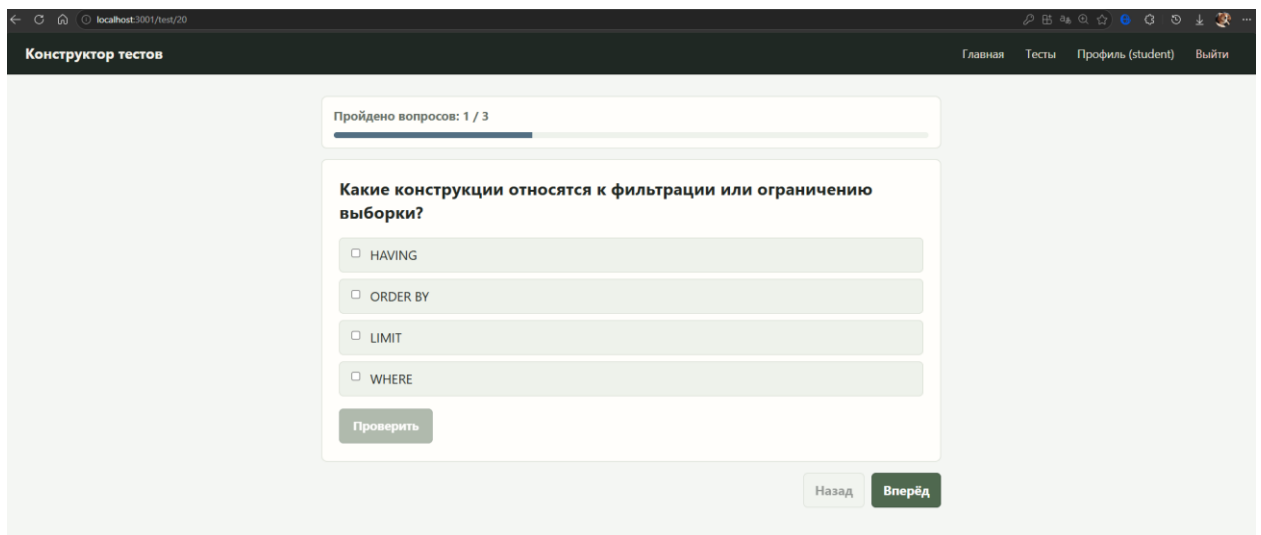


Рисунок 15 - Прохождение теста

После завершения теста приложение отображает количество правильных ответов, процент выполнения и текстовую оценку результата.

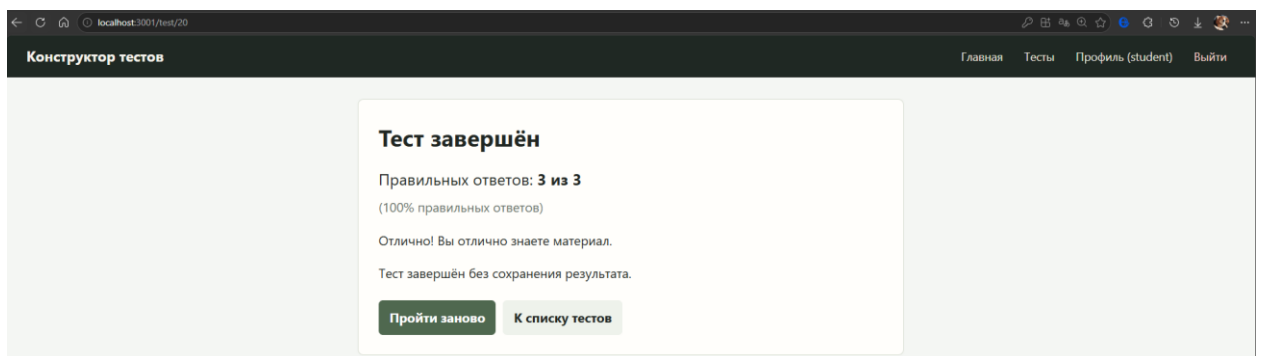


Рисунок 16 - Результат прохождения теста

8.2.3. Повтор ошибок

Если после завершения теста есть неправильные ответы, пользователь может повторить только ошибочные вопросы. Этот сценарий проверяет работу режима повтора ошибок и корректность получения вопросов, на которые пользователь ранее ответил неправильно.

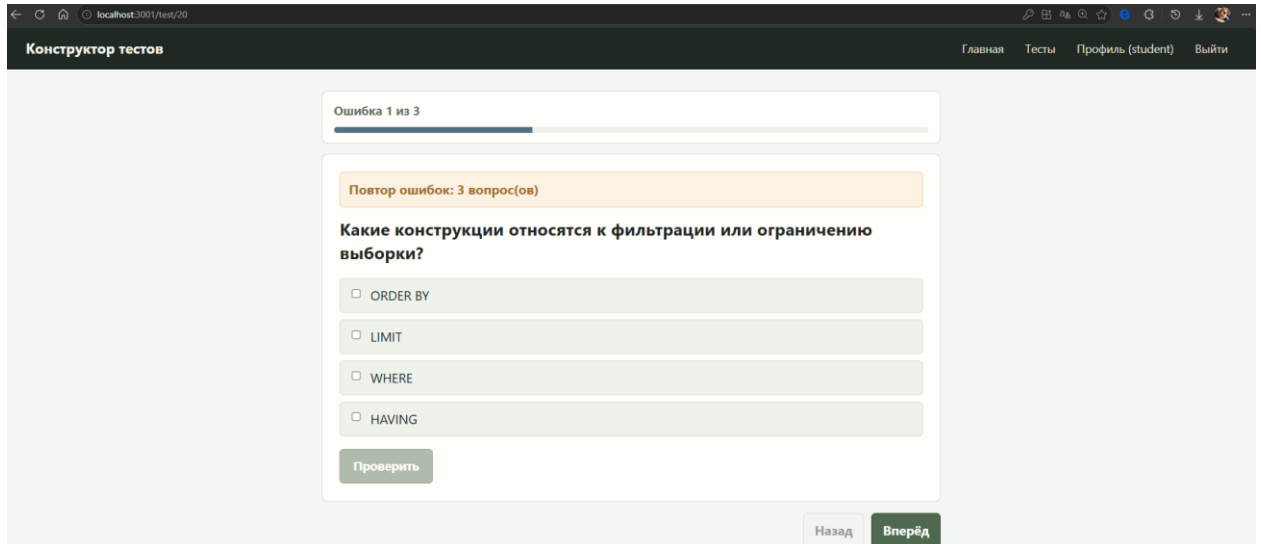


Рисунок 17 - Повтор ошибочных вопросов

8.2.4. Создание и редактирование теста

Для пользователя с ролью teacher или admin проверялась возможность открытия конструктора и создания нового теста.

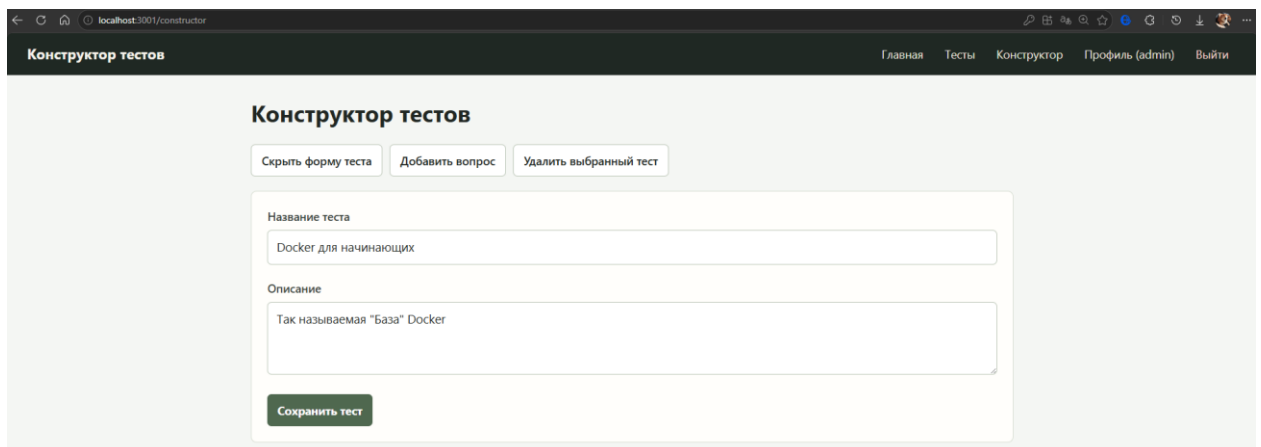


Рисунок 18 - Создание нового теста

Также проверялись редактирование названия, описания и удаление выбранного теста.

8.2.5. Добавление вопросов разных типов

В конструкторе проверялась возможность добавления вопросов трёх типов:

- вопрос с одним правильным ответом;
- вопрос с несколькими правильными ответами;
- вопрос на сопоставление.

The screenshot shows the 'Конструктор тестов' (Test Constructor) interface. At the top, there are three buttons: 'Создать тест' (Create test), 'Скрыть форму вопроса' (Hide question form), and 'Удалить выбранный тест' (Delete selected test). Below these is a dropdown menu 'Выберите тест:' (Select test:) with the value 'Docker для начинающих' (Docker for beginners). The main form is titled 'Тип вопроса' (Question type) and has a dropdown menu with the value 'Один правильный ответ' (One correct answer). Below this is a text input field 'Текст вопроса' (Question text) with the value 'Что такое Docker?'. Underneath is a section 'Варианты ответов' (Answer options) with four radio buttons and corresponding text boxes: 'Операционная система' (Operating system), 'Система управления базами данных' (Database management system), 'Платформа для контейнеризации приложений' (Application containerization platform), and 'Язык программирования' (Programming language). The third option is selected. To the right of each text box is a red 'Удалить' (Delete) button. At the bottom of the form are two buttons: 'Добавить вариант' (Add option) and 'Сохранить вопрос' (Save question).

Рисунок 19 - Добавление вопроса с одним правильным ответом

The screenshot shows the 'Конструктор тестов' (Test Constructor) interface. At the top, there are three buttons: 'Создать тест' (Create test), 'Скрыть форму вопроса' (Hide question form), and 'Удалить выбранный тест' (Delete selected test). Below these is a dropdown menu 'Выберите тест:' (Select test:) with the value 'Docker для начинающих' (Docker for beginners). The main form is titled 'Тип вопроса' (Question type) and has a dropdown menu with the value 'Несколько правильных ответов' (Several correct answers). Below this is a text input field 'Текст вопроса' (Question text) with the value 'Какие из перечисленных компонентов относятся к Docker?'. Underneath is a section 'Варианты ответов' (Answer options) with five checkboxes and corresponding text boxes: 'Dockerfile', 'Image (образ)', 'Container (контейнер)', 'Virtual Machine', and 'Kubernetes'. The first three options are checked. To the right of each text box is a red 'Удалить' (Delete) button. At the bottom of the form is a button 'Добавить вариант' (Add option).

Рисунок 20 - Добавление вопроса с несколькими правильными ответами

Конструктор тестов

Создать тест Скрыть форму вопроса Удалить выбранный тест

Выберите тест: Docker для начинающих

Тип вопроса
Сопоставление

Текст вопроса
Сопоставьте термин Docker с его описанием:

№	Левая часть	Правая часть	Соответствие	
1	Dockerfile	Запущенный экземпляр обра	Container	Удалить
2	Image	Инструкция для сборки обра	Dockerfile	Удалить
3	Container	Шаблон приложения	Image	Удалить

Добавить пару

Сохранить вопрос Сбросить форму

Рисунок 21 - Добавление вопроса на сопоставление

Для каждого типа вопроса проверялось, что форма не позволяет отправить некорректные данные: пустой текст вопроса, недостаточное количество вариантов, отсутствие правильного ответа или незаполненные пары для сопоставления.

8.3. Проверка ролевой модели

Отдельно проверялась работа ролевой модели. В приложении используются роли student, teacher и admin.

Студент должен иметь доступ только к просмотру и прохождению тестов. Преподаватель должен иметь доступ к конструктору и аналитике. Администратор должен иметь доступ к конструктору и административной панели.

На клиентской стороне проверялось отображение пунктов навигации. В компоненте Header доступ к конструктору открывается для teacher и admin, доступ к аналитике - только для teacher, а доступ к административной панели - только для admin.

На серверной стороне проверялось, что защищённые маршруты не выполняются без JWT-токена или с токеном пользователя без нужной роли.

Это важно, потому что скрытие кнопки в интерфейсе не является полноценной защитой: итоговая проверка доступа должна выполняться на backend.

8.4. Тестирование административной панели

Для неё проверялись следующие сценарии:

- доступ к панели только для пользователя с ролью admin;
- загрузка списка пользователей;
- изменение роли пользователя;
- удаление пользователя;
- загрузка списка тестов;
- включение и отключение активности теста;
- получение системных настроек;
- изменение названия платформы;
- включение и отключение публичной регистрации;
- включение и отключение модерации тестов преподавателей.

Административные маршруты проверялись как через интерфейс, так и через API. При изменении роли проверялось, что допускаются только значения student, teacher и admin. При изменении активности теста проверялось, что значение isActive является логическим. При изменении системных настроек проверялись типы и ограничения значений.

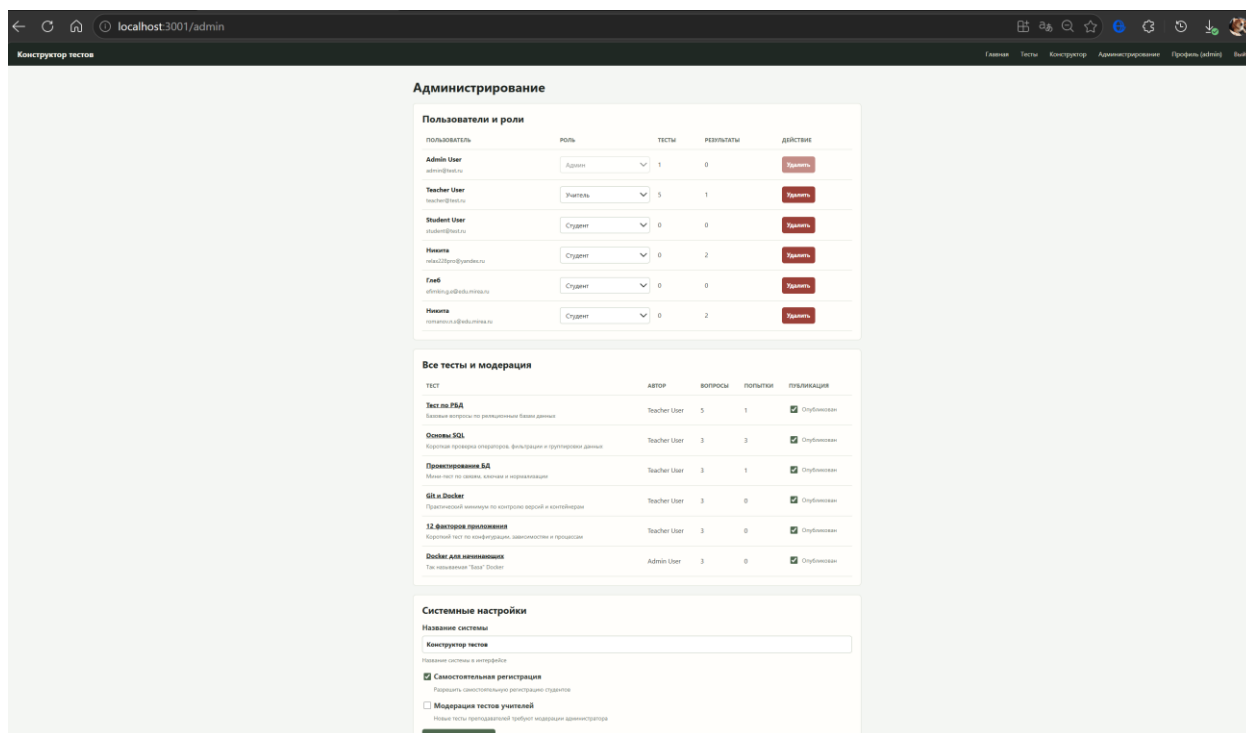


Рисунок 22 - Административная панель

8.5. Тестирование аналитики преподавателя

Для пользователя с ролью teacher проверялась страница аналитики результатов. Она загружает данные по прохождению тестов студентами и отображает сводные показатели: количество прохождений, количество студентов, количество тестов, средний результат и подробности по ответам.

Проверялись следующие сценарии:

- доступ к аналитике только для преподавателя;
- загрузка данных о прохождении;
- отображение результатов по студентам;
- отображение результатов по тестам;
- обработка ошибки загрузки данных;
- экспорт отчёта в PDF;
- экспорт отчёта в Excel.

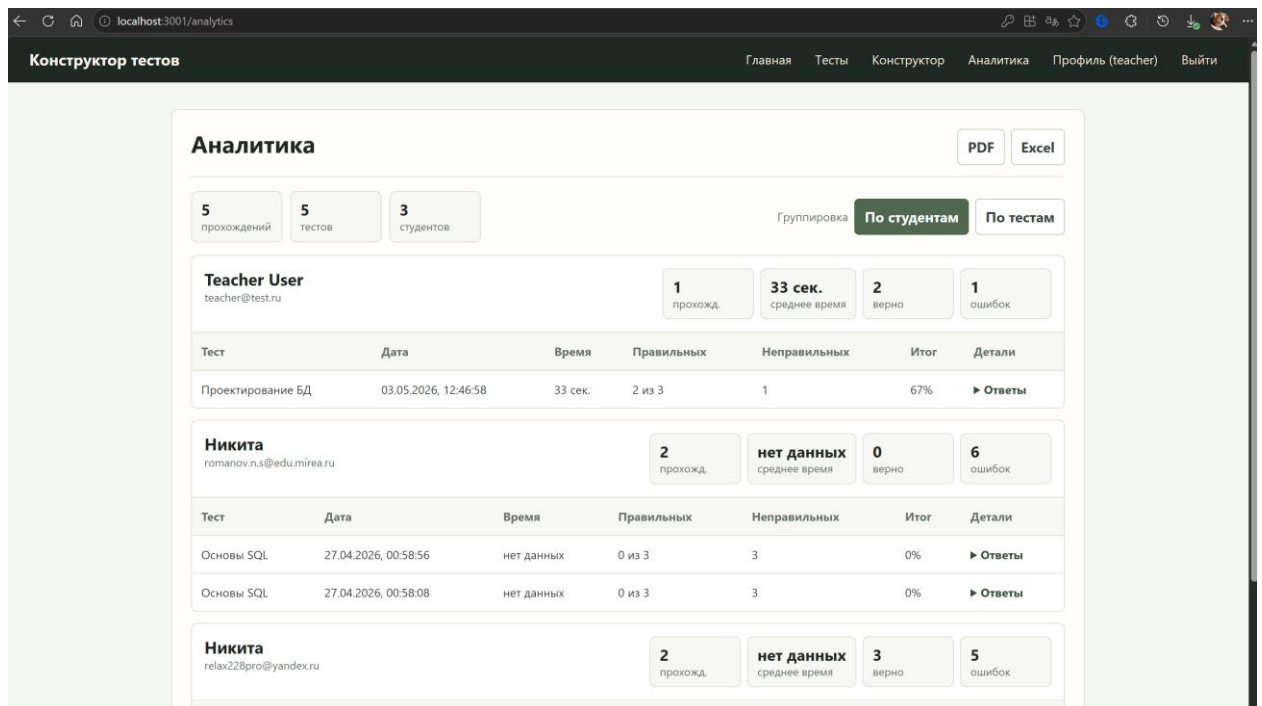


Рисунок 23 - Страница аналитики преподавателя

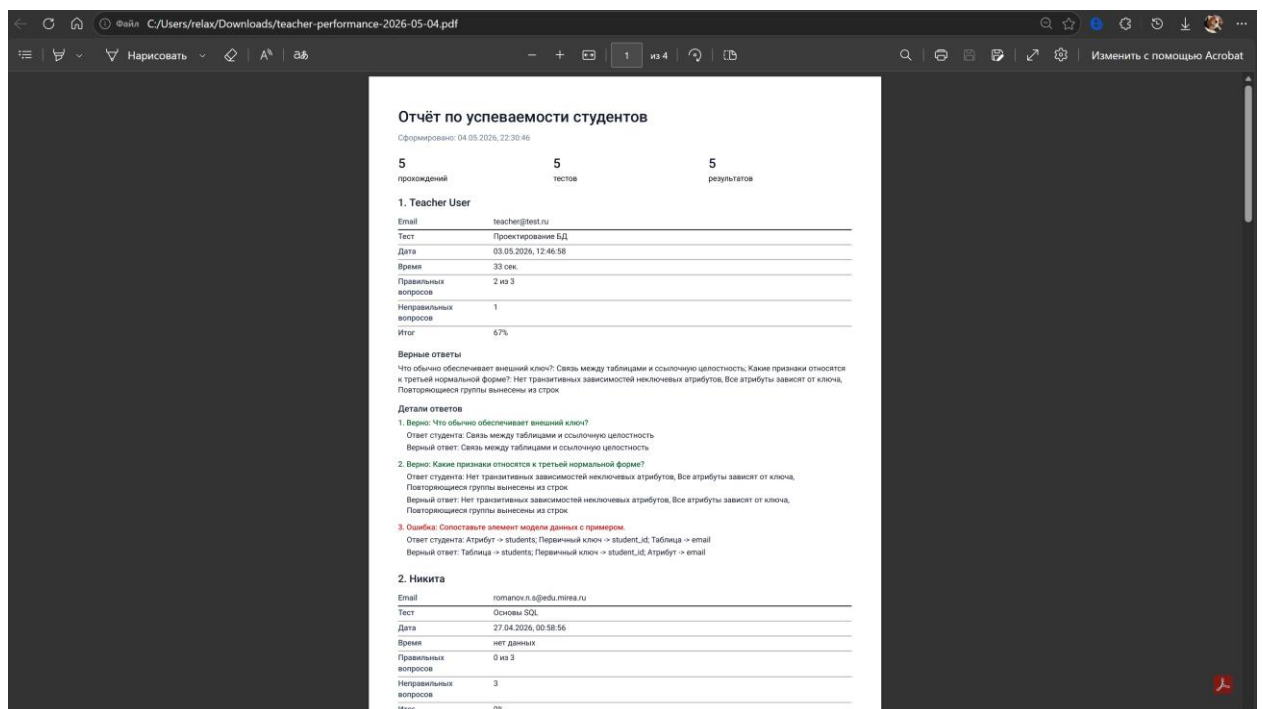


Рисунок 24 - Экспорт аналитики преподавателя в PDF

	A	B	C	D	E	F	G	H	I	J	K	L
	Студент	Email	Тест	Правильных вопро	Неправильных вои	Всего вопросов	Процент	Время	Дата			
2	Teacher User	teacher@test.ru	Проектирование	2	1	3		67 33 сек.	03.05.2026, 12:46:58			
3	Никита	romanov.n.s@edu	Основы SQL	0	3	3		0 нет данных	27.04.2026, 00:58:56			
4	Никита	romanov.n.s@edu	Основы SQL	0	3	3		0 нет данных	27.04.2026, 00:58:08			
5	Никита	relax228pro@yand	Тест по РБД	0	5	5		0 нет данных	26.04.2026, 13:19:22			
6	Никита	relax228pro@yand	Основы SQL	3	0	3		100 нет данных	26.04.2026, 13:19:07			

Рисунок 25 – Экспорт аналитики в Excel

8.6. Модульное и компонентное тестирование клиентской части

Для клиентской части используются Jest и React Testing Library. Тесты проверяют работу компонентов без ручного запуска всего приложения.

В проекте тестируются:

- маршрутизация приложения;
- отображение главной страницы;
- компонент Header и разделение навигации по ролям;
- страница списка тестов;
- форматирование количества вопросов и времени;
- компоненты прохождения теста;
- Redux-состояние авторизации;
- Redux-состояние прохождения теста;
- API-сервис и обработка ошибок;
- конструктор вопросов;
- административная панель;
- аналитика преподавателя.

Для тестов используется вспомогательный файл `test-utils/render.jsx`, который позволяет рендерить компоненты сразу с Redux-хранилищем и MemoryRouter. Это упрощает проверку страниц, зависящих от состояния пользователя и текущего маршрута.

Отдельно проверяется компонент QuestionsAdmin. В тестах создаётся тестовый Redux-store, имитируется пользователь с ролью преподавателя или администратора, мокируются API-запросы и проверяются действия в конструкторе.

```
PS C:\Users\relax\Documents\kursach_6_PiRKSP\client> npm run coverage

> test-constructor-client@1.0.0 coverage
> react-scripts test --coverage --watchAll=false --runInBand
```

```
PASS src/App.test.js
PASS src/pages/TeacherAnalytics/TeacherAnalytics.test.jsx
PASS src/pages/Admin/QuestionsAdmin.test.jsx
PASS src/pages/Admin/AdminDashboard.test.jsx
PASS src/components/Question/Question.test.jsx
PASS src/components/Quiz/QuizContainer.test.jsx
PASS src/pages/Register/Register.test.jsx
PASS src/pages/Profile/Profile.test.jsx
PASS src/pages/Login/Login.test.jsx
PASS src/pages/Test/Test.test.jsx
PASS src/components/Header/Header.test.jsx
PASS src/pages/Home/Home.test.jsx
PASS src/components/Score/Score.test.jsx
PASS src/components/QuizControls/QuizControls.test.jsx
PASS src/store/slices/quizSlice.test.js
PASS src/store/slices/authSlice.test.js
PASS src/services/api.test.js
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
src	100	100	100	100	
App.js	100	100	100	100	
src/components/Header	100	100	100	100	
Header.jsx	100	100	100	100	
src/components/Question	100	100	100	100	
Question.jsx	100	100	100	100	
src/components/Quiz	100	100	100	100	
QuizContainer.jsx	100	100	100	100	
src/components/QuizControls	100	100	100	100	
QuizControls.jsx	100	100	100	100	
src/components/Score	100	100	100	100	
Score.jsx	100	100	100	100	
src/pages/Admin	100	100	100	100	
AdminDashboard.jsx	100	100	100	100	
src/pages/Home	100	100	100	100	
Home.jsx	100	100	100	100	
src/pages/Login	100	100	100	100	
Login.jsx	100	100	100	100	
src/pages/Profile	100	100	100	100	
Profile.jsx	100	100	100	100	
src/pages/Register	100	100	100	100	
Register.jsx	100	100	100	100	
src/pages/TeacherAnalytics	100	100	100	100	
TeacherAnalytics.jsx	100	100	100	100	
src/pages/Test	100	100	100	100	
Test.jsx	100	100	100	100	
src/services	100	100	100	100	
api.js	100	100	100	100	
src/store	100	100	100	100	
store.js	100	100	100	100	
src/store/slices	100	100	100	100	
authSlice.js	100	100	100	100	
quizSlice.js	100	100	100	100	

```
Test Suites: 17 passed, 17 total
Tests: 82 passed, 82 total
Snapshots: 0 total
Time: 10.353 s, estimated 63 s
Ran all test suites.
```

Рисунок 26 - Успешное выполнение компонентных тестов клиентской части

8.7. Модульное тестирование серверной части

Для серверной части используются модульные тесты, проверяющие отдельные части backend-логики.

Проверялись следующие модули:

- middleware авторизации;

- middleware проверки ролей;
- валидация ВХОДНЫХ данных;
- логика оценки ответов;
- работа с ролями student, teacher, admin;
- системные настройки;
- Swagger-конфигурация;
- healthcheck-маршрут;
- маршруты результатов;
- ограничения для durationSeconds.

```

PS C:\Users\relax\Documents\kursach_6_PiRKSP\server> npm run coverage

> test-constructor-server@1.0.0 coverage
> node ../client/node_modules/jest/bin/jest.js --coverage --runInBand

PASS tests/swagger.test.js
PASS tests/middleware.test.js
PASS tests/systemSettings.test.js
PASS tests/roles.test.js
PASS tests/grading.test.js

-----
File                                % Stmts   % Branch   % Funcs   % Lines   Uncovered Line #s
-----
All files                          100       100       100       100
constants                          100       100       100       100
roles.js                           100       100       100       100
middleware                         100       100       100       100
auth.js                            100       100       100       100
optionalAuth.js                    100       100       100       100
role.js                            100       100       100       100
validate.js                        100       100       100       100
services                           100       100       100       100
systemSettings.js                  100       100       100       100
utils                              100       100       100       100
grading.js                         100       100       100       100
-----

Test Suites: 5 passed, 5 total
Tests:       29 passed, 29 total
Snapshots:   0 total
Time:        1.633 s, estimated 2 s
Ran all test suites.

```

Рисунок 27 - Успешное выполнение тестов серверной части

8.8. Проверка Swagger и healthcheck

В новой версии backend предоставляет документацию API через Swagger/OpenAPI. Проверялась доступность Swagger UI и JSON-описания API.

Проверяемые маршруты:

- GET /api/docs;
- GET /api/docs.json;

– GET /api/health.

Маршрут /api/health используется для проверки работоспособности backend. Он применяется не только при ручной проверке, но и в CI/CD, где после запуска контейнеров выполняется ожидание доступности backend по адресу `http://localhost:5000/api/health`.

```
PS C:\Users\relax\Documents\kursach_6_PiRKSP> curl http://localhost:5000/api/health

StatusCode      : 200
StatusDescription : OK
Content         : {"status":"OK","timestamp":"2026-05-04T19:43:28.482Z","environment":"development"}
RawContent      : HTTP/1.1 200 OK
                  Access-Control-Allow-Origin: *
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Content-Length: 82
                  Content-Type: application/json; charset=utf-8
                  Date: Mon, 04 May 2026 19:43:28 GMT
                  ...
Forms           : {}
Headers        : {[Access-Control-Allow-Origin, *], [Connection, keep-alive], [Keep-Alive, timeout=5], [Content-Length, 82]...}
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     : mshtml.HTMLDocumentClass
RawContentLength : 82
```

Рисунок 28 - Проверка healthcheck-маршрута backend

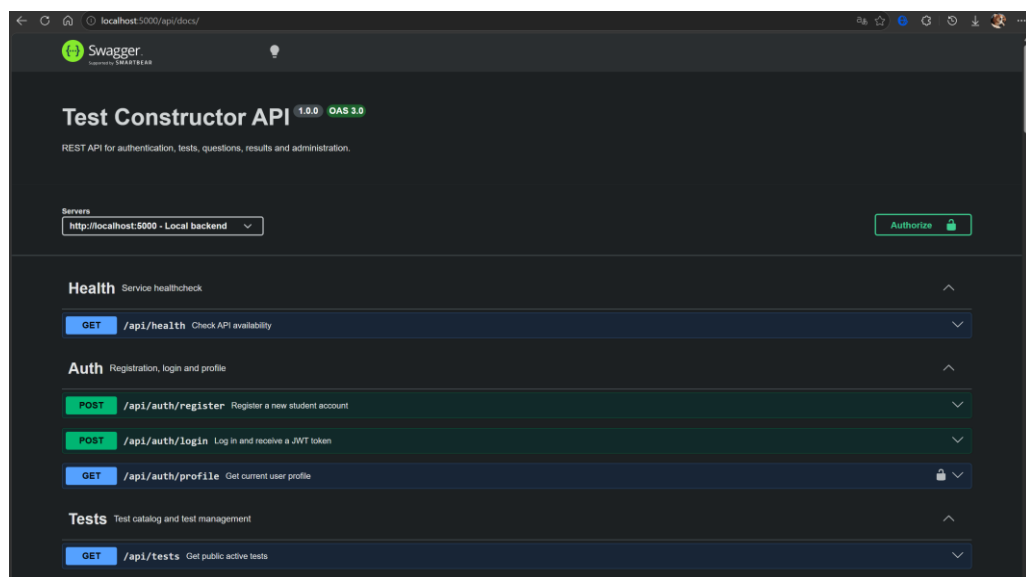


Рисунок 29 - Swagger-документация API

8.9. Фаззинг-тестирование API

Фаззинг-тестирование проводилось для проверки устойчивости серверной части к некорректным, случайным и потенциально опасным входным данным. Целью было убедиться, что backend не завершает работу аварийно и возвращает корректные HTTP-ответы с ошибками.

Перед запуском фаззинга выполняется проверка доступности API через

/api/health. Если backend недоступен, скрипт выводит сообщение о необходимости запустить сервер или Docker Compose-сервис.

Проверялись следующие группы данных:

- неверный формат email;
- слишком короткий пароль;
- попытка регистрации с ролью admin;
- пустое название теста;
- неправильный тип вопроса;
- пустой массив вариантов ответа;
- повреждённый JSON;
- неверный JWT-токен;
- отсутствие JWT-токена;
- отрицательные или строковые идентификаторы;
- SQL Injection;
- XSS payloads;
- path traversal;
- command injection;
- null bytes;
- слишком большие строки;
- нарушение ролевого доступа.

В fuzzing-скриптах отдельно проверяются SQL-инъекции через /api/auth/login и маршруты тестов, XSS-полезные нагрузки через регистрацию, некорректные заголовки авторизации для защищённых маршрутов, path traversal, command injection и null bytes.

```

PS C:\Users\relax\Documents\kursach_6_PiRKSP> node .\server\tests\fuzzing\fuzzing.js
Starting fuzzing tests...

Target: http://localhost:5000

Testing SQL injection...

PASS SQL Injection Login -> /api/auth/login (400)
PASS SQL Injection Route -> /api/tests/'%20OR%20'1'%3D'1 (400)
PASS SQL Injection Login -> /api/auth/login (400)
PASS SQL Injection Route -> /api/tests/'%3B%20DROP%20TABLE%20users%3B-- (400)
PASS SQL Injection Login -> /api/auth/login (400)
PASS SQL Injection Route -> /api/tests/'%20UNION%20SELECT%20*%20FROM%20users-- (400)
PASS SQL Injection Login -> /api/auth/login (401)
PASS SQL Injection Route -> /api/tests/admin'-- (400)
PASS SQL Injection Login -> /api/auth/login (400)
PASS SQL Injection Route -> /api/tests/'%20OR%201%3D1-- (400)

Testing XSS payload handling...

PASS XSS Register Payload -> /api/auth/register (400)
PASS XSS Register Payload -> /api/auth/register (400)
PASS XSS Register Payload -> /api/auth/register (400)

Testing authentication guards...

PASS Protected Profile Access -> /api/auth/profile (401)
PASS Protected Profile Access -> /api/auth/profile (401)
PASS Protected Profile Access -> /api/auth/profile (401)
PASS Protected Profile Access -> /api/auth/profile (401)

Testing path traversal, command injection and null bytes...

PASS Traversal/Command Route Probe -> /api/tests/..%2F..%2F..%2Fetc%2Fpasswd/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/..%5C..%5C..%5Cwindows%5Csystem32%5Cconfig%5Csam/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%252e%252e%252f%252e%252e%252f%252e%252e%252fetc%252fpasswd/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%3B%20cat%20%2Fetc%2Fpasswd/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%26%26%20whoami/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%24(id)/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%00/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/test%00admin/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%2500/questions (400)

Testing large payloads...

PASS Large Payload Register -> /api/auth/register (400)
PASS Large Payload Register -> /api/auth/register (400)
PASS Large Payload Register -> /api/auth/register (400)

Testing RBAC...

PASS Auth Login -> /api/auth/login (200)
PASS Auth Login -> /api/auth/login (200)
PASS Auth Login -> /api/auth/login (200)
PASS RBAC Create Test (admin) -> /api/tests (201)
PASS RBAC Delete Test (admin) -> /api/tests/42 (200)
PASS RBAC Create Test (teacher) -> /api/tests (201)
PASS RBAC Delete Test (teacher) -> /api/tests/43 (200)
PASS RBAC Create Test (student) -> /api/tests (403)

Results saved to: C:\Users\relax\Documents\kursach_6_PiRKSP\server\tests\fuzzing\results\fuzzing-report-1777923996527.json
Passed checks: 37
Failed checks: 0

```

Рисунок 30 - Успешное выполнение фаззинг-тестирования API

8.10. Проверка Docker Compose

Проверялись следующие сервисы: postgres, backend, frontend, nginx.

Для PostgreSQL используется healthcheck через pg_isready, а backend запускается только после успешной проверки базы данных. Backend работает

на порту 5000, frontend - на порту 3001 в локальной конфигурации, а Nginx принимает запросы на порту 80.

Проверялись следующие действия:

- сборка контейнеров;
- запуск всех сервисов;
- подключение backend к PostgreSQL;
- доступность frontend;
- проксирование /api/ через Nginx;
- доступность /api/health через backend;
- доступность /api/health через Nginx.

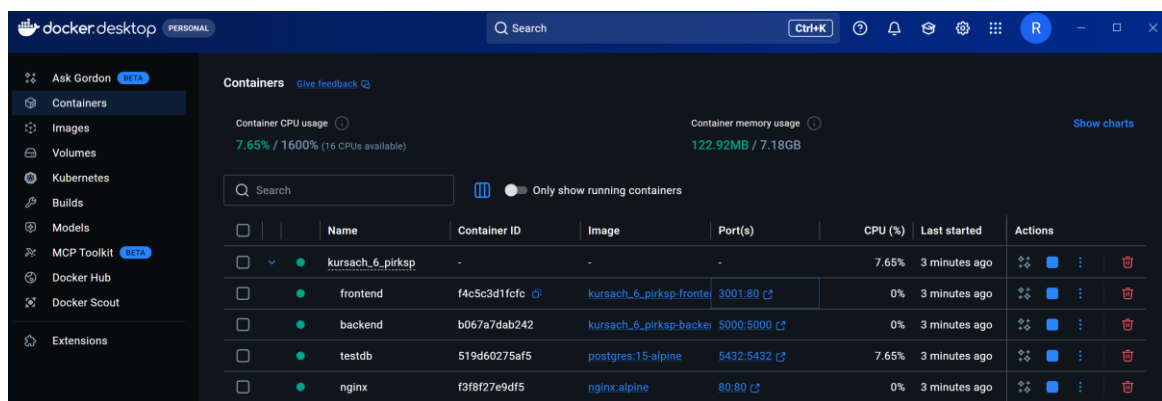


Рисунок 31 - Запущенные контейнеры Docker Compose

8.11. CI/CD-тестирование

В новой версии проекта настроен CI/CD-процесс через GitHub Actions. Workflow запускается при push, pull_request в ветку main, а также вручную через workflow_dispatch.

CI/CD состоит из трёх основных этапов.

На первом этапе выполняются тесты. GitHub Actions получает код из репозитория, устанавливает Node.js 18, устанавливает зависимости клиентской и серверной частей через `npm ci --prefix client` и `npm ci --prefix server`, после чего запускает общую команду `npm test`.

На втором этапе проверяется контейнеризированный запуск. Workflow собирает и запускает Docker Compose stack, затем ожидает доступности backend, frontend и Nginx-прокси. Для backend проверяется

http://localhost:5000/api/health, для frontend - http://localhost:3001, для Nginx - http://localhost/api/health. При ошибке выводятся логи Docker Compose.

На третьем этапе выполняется публикация Docker-образов в Docker Hub. Если событие не является pull request, workflow авторизуется в Docker Hub и публикует образы backend и frontend с тегами latest и SHA текущего коммита.

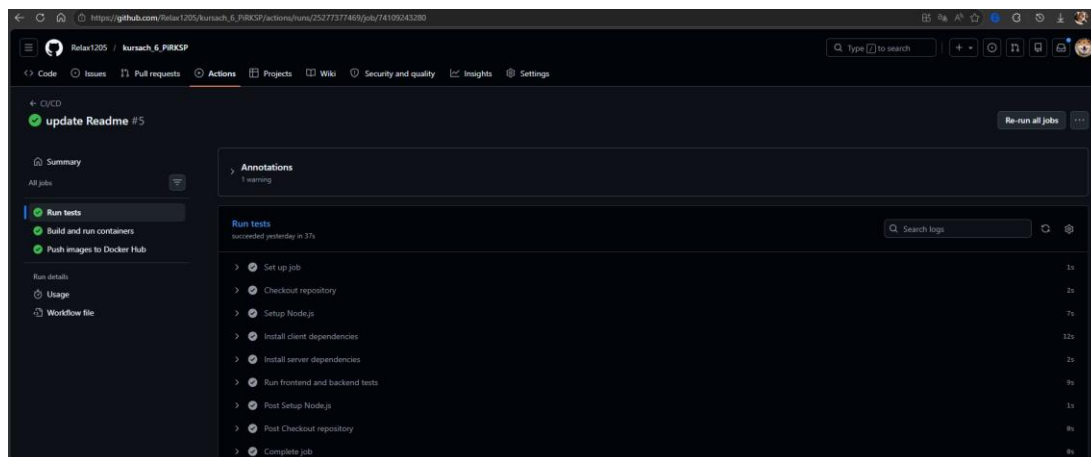


Рисунок 32 - Успешное выполнение GitHub Actions

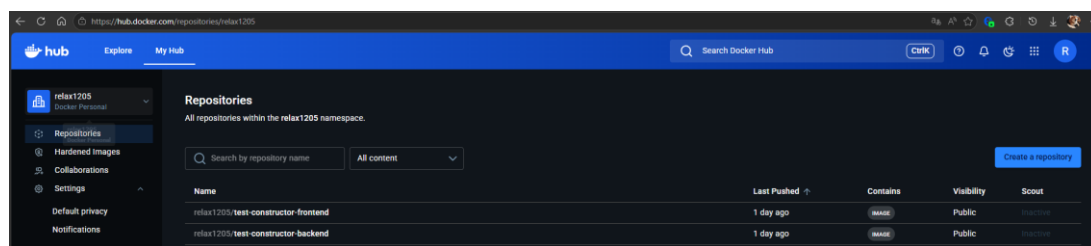


Рисунок 33 - Публикация Docker-образов в Docker Hub

8.12. Итог тестирования

В результате тестирования были проверены основные пользовательские сценарии: регистрация, авторизация, просмотр тестов, прохождение теста, повтор ошибок, создание тестов, добавление вопросов, административная панель, управление пользователями и ролями, управление активностью тестов, системные настройки, аналитика преподавателя, экспорт отчётов, Swagger-документация, healthcheck-маршрут, Docker Compose и CI/CD.

Проведённые проверки показали, что приложение корректно обрабатывает основные сценарии работы, ограничивает доступ по ролям, валидирует входные данные, устойчиво реагирует на некорректные запросы и может быть автоматически проверено перед развёртыванием.

9. Размещение проекта в Git

В рамках выполнения курсовой работы разработанное клиент-серверное приложение было оформлено в виде проекта и размещено в системе контроля версий Git.

Исходный код проекта размещён в удалённом репозитории GitHub:

Ссылка на репозиторий: https://github.com/Relax1205/kursach_6_PiRKSP

Использование Git позволило организовать хранение исходного кода, вести историю изменений, разделять разработку по веткам и подготовить проект к автоматизированной проверке и развёртыванию.

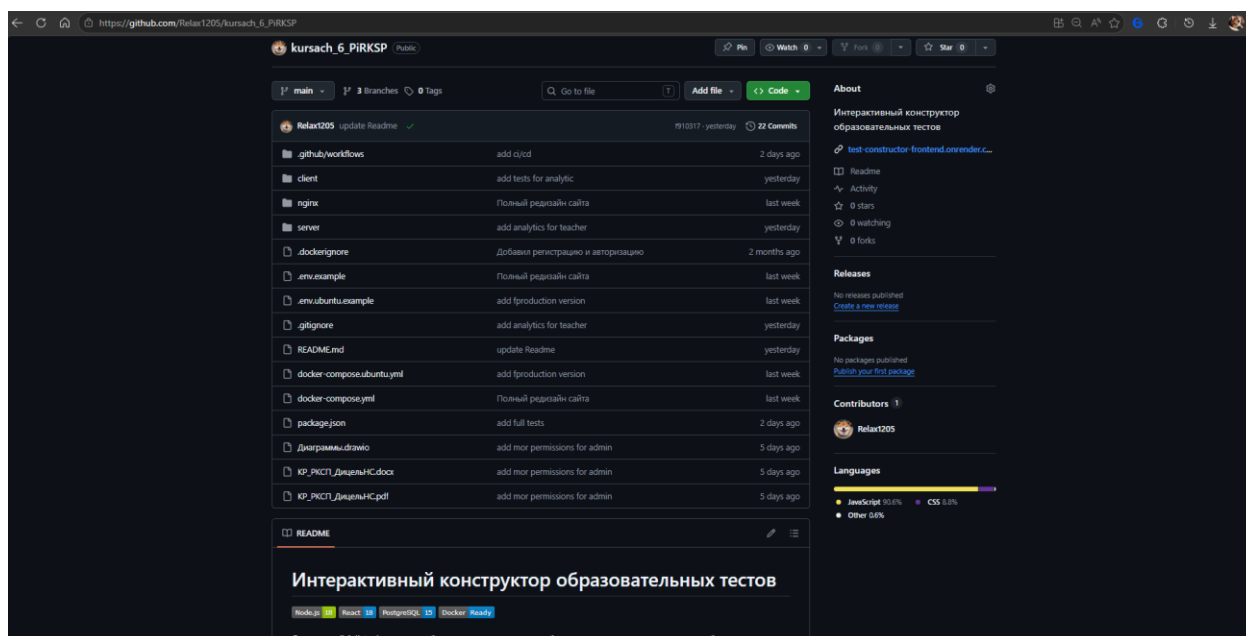


Рисунок 34 - Репозиторий проекта на GitHub

9.1. Организация веток в репозитории

В репозитории использовались несколько веток, каждая из которых выполняла отдельную роль в процессе разработки.

Основной веткой является main. В ней хранится стабильная версия проекта, объединяющая готовый функционал клиентской и серверной частей. Эта ветка используется как основная для хранения итоговой версии приложения.

Ветка feature использовалась для разработки нового функционала. В ней реализовывались и проверялись изменения, связанные с расширением

возможностей приложения: административная панель, аналитика преподавателя, системные настройки, обновление интерфейса и доработка серверных маршрутов. После проверки изменения могли быть перенесены в основную ветку.

Ветка tests использовалась для разработки и доработки тестов. В ней добавлялись компонентные тесты клиентской части, серверные тесты, проверки ролевой модели, API, Swagger, healthcheck и других новых возможностей приложения.

Такое разделение позволило не вносить экспериментальные изменения сразу в основную ветку и отдельно работать над функционалом и тестированием.

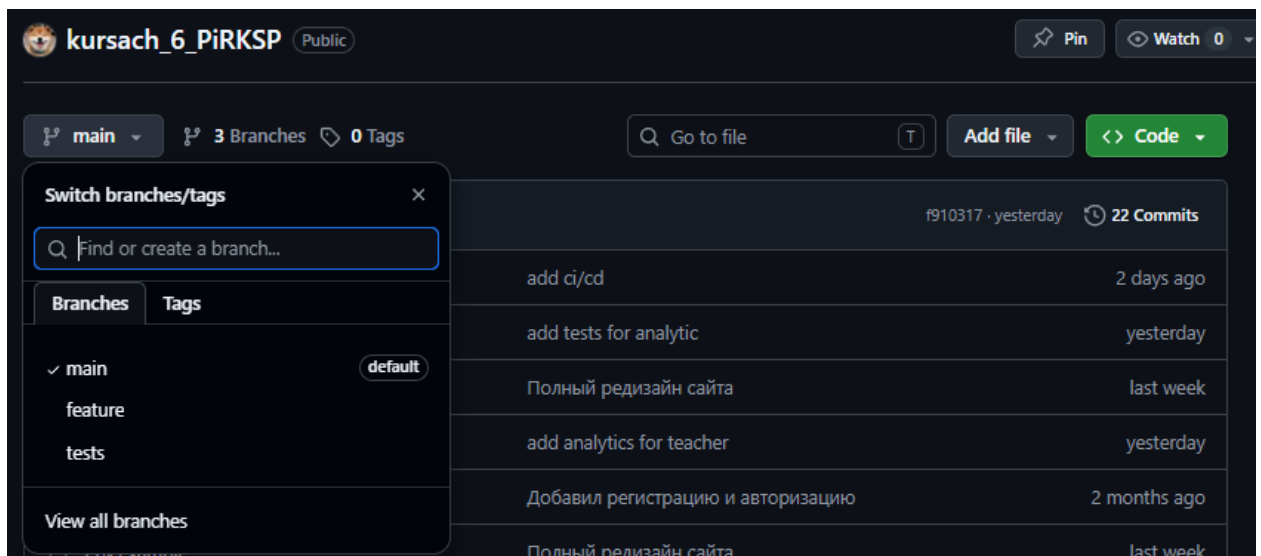


Рисунок 35 - Ветки репозитория main, feature, tests

9.2. Использование Git в процессе разработки

В процессе работы Git использовался для:

- хранения и версионирования исходного кода;
- фиксации этапов разработки через коммиты;
- разделения стабильной версии, нового функционала и тестов по веткам;
- отслеживания изменений в клиентской и серверной частях;
- хранения конфигурационных файлов Docker, Docker Compose, Nginx и GitHub Actions;

- подготовки проекта к автоматизированной проверке и развёртыванию.

Структура проекта в репозитории включает клиентскую часть `client`, серверную часть `server`, конфигурации `Docker Compose`, переменные окружения-примеры, настройки `Nginx` и `workflow` для `CI/CD`.

9.3. Подключение CI/CD

К репозиторию подключён `CI/CD`-процесс через `GitHub Actions`. Для этого в проекте добавлен `workflow .github/workflows/ci-cd.yml`. Он запускается при отправке изменений в ветки `main` и `master`, при создании `pull request` в эти ветки, а также вручную через `workflow_dispatch`.

`CI/CD`-процесс состоит из нескольких этапов.

Сначала выполняется установка зависимостей клиентской и серверной частей через `npm ci --prefix client` и `npm ci --prefix server`. После этого запускаются тесты `frontend` и `backend` командой `npm test`.

Затем `GitHub Actions` собирает и запускает `Docker Compose stack`. После запуска выполняется проверка доступности `backend` через `http://localhost:5000/api/health`, `frontend` через `http://localhost:3001` и `Nginx`-прокси через `http://localhost/api/health`. Если проверка завершается ошибкой, `workflow` выводит логи контейнеров для диагностики.

После успешного прохождения тестов и проверки контейнеров `workflow` может публиковать `Docker`-образы `backend` и `frontend` в `Docker Hub`. Для этого используются секреты `DOCKERHUB_USERNAME` и `DOCKERHUB_TOKEN`, а образы публикуются с тегами `latest` и `SHA` текущего коммита.

9.4. Этапы CI/CD-пайплайна

Для наглядного описания автоматизированной проверки проекта `CI/CD`-процесс можно представить как последовательность этапов. Каждый этап выполняется в `GitHub Actions` и зависит от успешного завершения предыдущего. Такой подход позволяет не допустить публикацию или развёртывание версии приложения, если тесты или контейнерная сборка завершились с ошибкой.

Таблица 18 - Этапы CI/CD-пайплайна

Этап	Действие
Checkout	Получение исходного кода из GitHub-репозитория
Setup	Установка Node.js и подготовка окружения
Install	Установка зависимостей клиентской и серверной частей
Test	Запуск автоматизированных тестов frontend и backend
Build	Сборка и запуск Docker Compose stack
Healthcheck	Проверка доступности backend, frontend и Nginx
Logs	Вывод логов контейнеров при ошибке сборки или запуска
Cleanup	Остановка Docker Compose stack после проверки
Publish	Публикация Docker-образов frontend и backend в Docker Hub

На первом этапе GitHub Actions получает код из репозитория и подготавливает окружение Node.js. Затем отдельно устанавливаются зависимости клиентской и серверной частей с помощью команд `npm ci --prefix client` и `npm ci --prefix server`. После установки зависимостей запускаются автоматизированные тесты командой `npm test`.

После успешного прохождения тестов выполняется сборка и запуск контейнеров через Docker Compose. На этом этапе проверяется, что приложение может быть запущено в контейнеризированной среде вместе с базой данных, frontend, backend и Nginx.

Для проверки работоспособности сервисов используются healthcheck-запросы. Backend проверяется по адресу `http://localhost:5000/api/health`, frontend - по адресу `http://localhost:3001`, а работа Nginx-прокси - по адресу `http://localhost/api/health`. Если один из сервисов недоступен, workflow завершится с ошибкой и выведет логи контейнеров для диагностики.

Финальным этапом является публикация Docker-образов в Docker Hub. Она выполняется после успешной сборки и проверки контейнеров. Для авторизации используются секреты `DOCKERHUB_USERNAME` и `DOCKERHUB_TOKEN`, а опубликованные образы получают теги `latest` и тег с SHA текущего коммита.

10. Контейнеризация и развёртывание

Для упрощения запуска и развёртывания приложения используется Docker. Все основные компоненты системы запускаются в отдельных контейнерах и объединяются в общую сеть. Это позволяет запускать приложение одинаково на локальной машине, сервере или в CI/CD-среде.

В проекте контейнеризованы следующие компоненты: frontend, backend, PostgreSQL и Nginx. Для локального запуска используется файл docker-compose.yml, используется общая сеть app-network, через которую контейнеры взаимодействуют друг с другом.

10.1. Dockerfile клиентской части

Клиентская часть собирается с помощью Dockerfile, расположенного в директории client.

Сборка frontend выполняется в два этапа. На первом этапе используется Node.js-образ, устанавливаются зависимости и выполняется сборка React-приложения. На втором этапе собранные статические файлы переносятся в контейнер Nginx, который отдаёт готовое приложение пользователю.

При сборке frontend используется аргумент REACT_APP_API_URL. Он нужен для того, чтобы React-приложение знало адрес backend-сервиса. Для локального запуска используется адрес локального backend, а для production - адрес backend, размещённого на Render.

```
client > Dockerfile
1  FROM node:18-alpine AS build
2
3  WORKDIR /app
4
5  COPY package*.json ./
6  RUN npm ci
7
8  COPY . .
9
10 ARG REACT_APP_API_URL
11 ENV REACT_APP_API_URL=$REACT_APP_API_URL
12
13 RUN npm run build
14
15 FROM nginx:alpine
16
17 COPY nginx.conf /etc/nginx/conf.d/default.conf
18
19 COPY --from=build /app/build /usr/share/nginx/html
```

Рисунок 36 - Dockerfile клиентской части

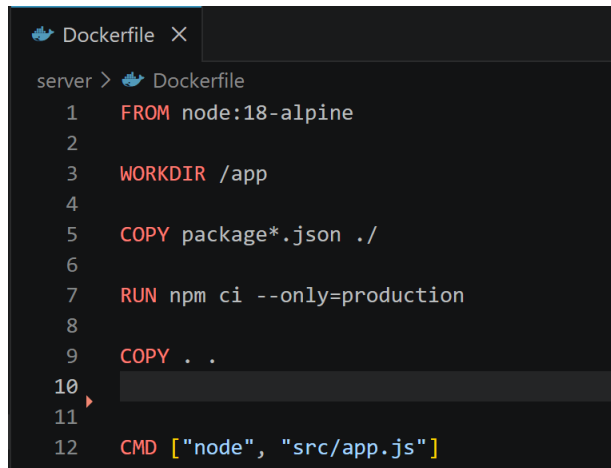
10.2. Dockerfile серверной части

Серверная часть запускается в отдельном Node.js-контейнере. Dockerfile backend расположен в директории server.

Основные этапы сборки backend:

1. установка зависимостей;
2. копирование исходного кода;
3. запуск серверного приложения через `npm start`.

Backend получает параметры работы через переменные окружения: `PORT`, `DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRE` и `NODE_ENV`. Благодаря этому один и тот же контейнер можно запускать как локально, так и в production-среде с разными настройками.

A screenshot of a code editor showing a Dockerfile. The file is named 'Dockerfile' and is located in the 'server' directory. The content of the Dockerfile is as follows:

```
1 FROM node:18-alpine
2
3 WORKDIR /app
4
5 COPY package*.json ./
6
7 RUN npm ci --only=production
8
9 COPY . .
10
11
12 CMD ["node", "src/app.js"]
```

Рисунок 37 - Dockerfile серверной части

10.3. Docker Compose

Для одновременного запуска всех контейнеров используется Docker Compose. В локальной конфигурации `docker-compose.yml` определены четыре сервиса:

- postgres - база данных PostgreSQL;
- backend - серверная часть на Node.js и Express;
- frontend - собранное React-приложение;
- nginx - reverse proxy.

Для локального запуска используется команда: `docker compose up -d --build`.

В локальной среде PostgreSQL доступен на порту 5432, backend - на

порту 5000, frontend - на порту 3001, а Nginx - на порту 80. Backend запускается только после успешной проверки готовности PostgreSQL через healthcheck.

10.4. PostgreSQL-контейнер

База данных запускается в отдельном контейнере на основе образа postgres:15-alpine.

Для хранения данных используется volume postgres_data, благодаря чему данные не теряются после перезапуска контейнера. Также подключается файл server/init.sql, который используется для начальной инициализации базы данных.

Подключение backend к базе данных выполняется через переменную окружения DATABASE_URL. В Docker Compose она формируется на основе переменных POSTGRES_USER, POSTGRES_PASSWORD и POSTGRES_DB.

Пример структуры строки подключения:

```
DATABASE_URL=postgresql://POSTGRES_USER:POSTGRES_PASSWORD@postgres:5432/POSTGRES_DB
```

10.5. Nginx как reverse proxy

Nginx используется как промежуточный сервер между пользователем, frontend и backend.

Он выполняет две основные функции: отдаёт статические файлы frontend и перенаправляет API-запросы к backend. Запросы к пользовательскому интерфейсу обрабатываются frontend-сервисом, а запросы с префиксом /api/ проксируются на backend.

Такой подход позволяет пользователю обращаться к одному адресу приложения, а внутри Docker-сети запросы распределяются между нужными контейнерами.

```

1  server {
2      listen 80;
3      server_name localhost;
4
5      location / {
6          proxy_pass http://frontend:80;
7          proxy_http_version 1.1;
8          proxy_set_header Upgrade $http_upgrade;
9          proxy_set_header Connection 'upgrade';
10         proxy_set_header Host $host;
11         proxy_cache_bypass $http_upgrade;
12         proxy_set_header X-Real-IP $remote_addr;
13         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
14     }
15
16     location /api/ {
17         proxy_pass http://backend:5000;
18         proxy_http_version 1.1;
19         proxy_set_header X-Real-IP $remote_addr;
20         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
21         proxy_set_header Host $http_host;
22         proxy_set_header X-NginX-Proxy true;
23         proxy_set_header Upgrade $http_upgrade;
24         proxy_set_header Connection "upgrade";
25         proxy_redirect off;
26         proxy_buffering off;
27     }
28
29     add_header X-Frame-Options "SAMEORIGIN" always;
30     add_header X-Content-Type-Options "nosniff" always;
31     add_header X-XSS-Protection "1; mode=block" always;
32     add_header Strict-Transport-Security "max-age=31536000" always;
33 }

```

Рисунок 38 - Конфигурация Nginx

10.6. Публикация Docker-образов в Docker Hub

После успешного прохождения тестов и проверки контейнеров workflow авторизуется в Docker Hub с использованием секретов DOCKERHUB_USERNAME и DOCKERHUB_TOKEN. Затем отдельно собираются и публикуются Docker-образы backend и frontend. Для каждого образа используются два тега: latest и тег с SHA текущего коммита.

Публикуемые образы имеют следующую структуру имён:

<DOCKERHUB_USERNAME>/test-constructor-backend:latest

<DOCKERHUB_USERNAME>/test-constructor-frontend:latest

Для frontend при сборке образа также передаётся аргумент REACT_APP_API_URL, чтобы собранное приложение обращалось к нужному backend-сервису.

10.7. Развёртывание на Render

Помимо Docker-развёртывания, приложение размещено на хостинге Render. Развёртывание выполнено по клиент-серверной схеме: frontend и backend работают как отдельные сервисы.

Frontend размещён по адресу: <https://test-constructor-frontend.onrender.com>

Backend размещён по адресу: <https://test-constructor-backend.onrender.com>

Для frontend на Render задаётся переменная окружения:

REACT_APP_API_URL=https://test-constructor-backend.onrender.com

Она указывает клиентскому приложению, куда отправлять API-запросы.

Для backend на Render используются переменные окружения: PORT, NODE_ENV, DATABASE_URL, JWT_SECRET, JWT_EXPIRE, FRONTEND_URL, CORS_ORIGINS.

Переменная DATABASE_URL используется для подключения к облачной PostgreSQL-базе данных. Переменная CORS_ORIGINS ограничивает обращения к backend со стороны разрешённого frontend-адреса.

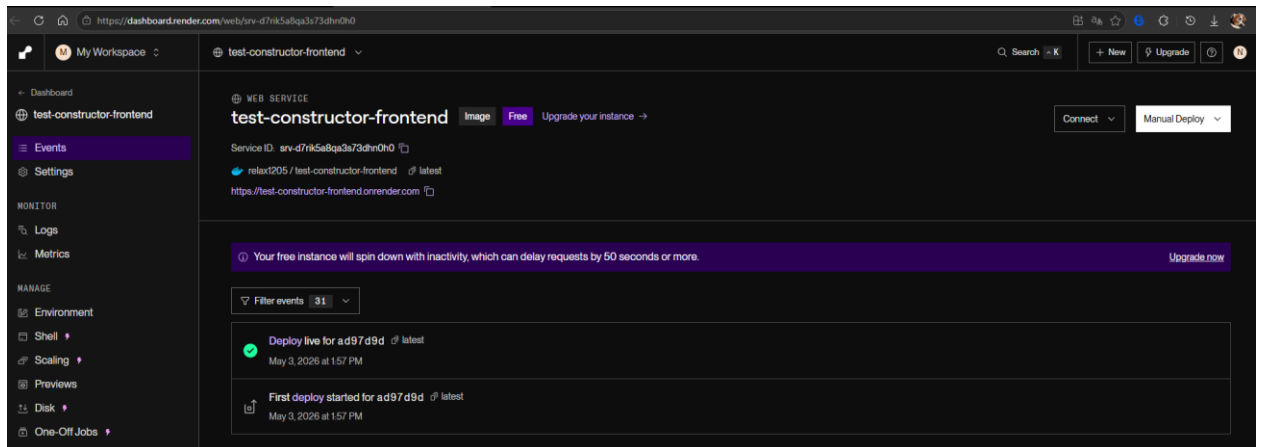


Рисунок 39 - Развёртывание frontend на Render

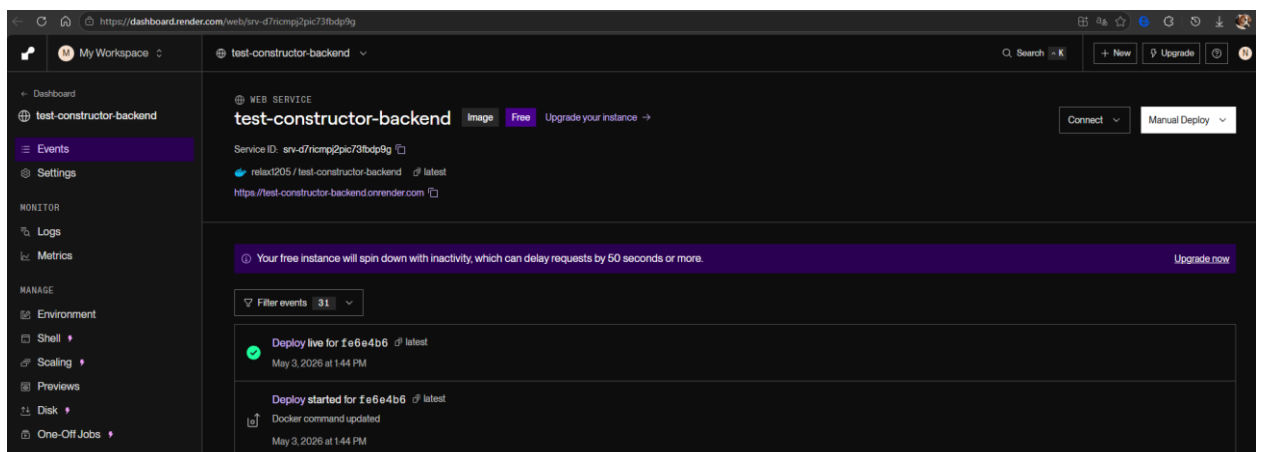


Рисунок 40 - Развёртывание backend на Render

Заключение

В рамках выполнения курсовой работы была разработана система «Интерактивный конструктор образовательных тестов», представляющая собой клиент-серверное веб-приложение для создания, прохождения, администрирования и анализа результатов образовательных тестов.

В ходе работы была подтверждена актуальность выбранной темы, связанная с развитием цифровых образовательных технологий и необходимостью автоматизации контроля знаний обучающихся.

В процессе выполнения курсовой работы были решены поставленные задачи. Проведён анализ предметной области и существующих решений, определены основные пользователи системы, описаны сценарии использования, сформулированы функциональные, нефункциональные требования, требования к безопасности и ролевой модели доступа.

Была разработана архитектура приложения на основе клиент-серверного подхода. Клиентская часть реализована как React SPA, серверная часть - как REST API на Node.js и Express.js, а для хранения данных используется PostgreSQL. Для работы с базой данных применён Sequelize.

В клиентской части реализованы основные пользовательские страницы: регистрация, авторизация, профиль, список тестов, прохождение теста, конструктор вопросов, аналитика преподавателя и административная панель. Для управления состоянием используется Redux Toolkit, а взаимодействие с сервером выполняется через API-сервис на основе Axios.

В серверной части реализованы регистрация и авторизация пользователей с использованием JWT, проверка ролей, валидация входных данных, CRUD-операции для тестов и вопросов, сохранение результатов прохождения, повтор ошибок, административные маршруты, системные настройки, аналитика преподавателя, Swagger-документация API и healthcheck-маршрут.

В проекте была реализована ролевая модель. Студент может проходить тесты и просматривать результаты, преподаватель - создавать тесты,

управлять вопросами и анализировать результаты прохождения, администратор - управлять пользователями, ролями, активностью тестов и системными настройками. Также реализованы экспорт аналитики преподавателя в PDF и Excel, ограничение количества вопросов, учёт времени прохождения теста и возможность включать или отключать активность тестов.

Проведено тестирование приложения. Были проверены основные пользовательские сценарии, работа клиентских компонентов, Redux-состояния, API-сервиса, серверных middleware, ролевой модели, административной панели, аналитики преподавателя, Swagger-документации и healthcheck-маршрута. Также выполнено фаззинг-тестирование API.

Проект размещён в системе контроля версий GitHub. В репозитории использовались несколько веток: main для стабильной версии проекта, feature для разработки нового функционала и tests для реализации и проверки тестов. К репозиторию подключён CI/CD-процесс через GitHub Actions.

Для упрощения запуска и развёртывания приложения использованы Docker и Docker Compose. Компоненты системы запускаются в отдельных контейнерах: frontend, backend, PostgreSQL и Nginx. Также настроен Nginx как reverse proxy. Дополнительно реализована публикация Docker-образов frontend и backend в Docker Hub.

Итоговая версия приложения размещена на хостинге Render. Frontend и backend развёрнуты как отдельные сервисы, а подключение клиентской части к серверной выполняется через production-переменные окружения. Это позволяет использовать приложение не только локально, но и через публичный веб-адрес.

Таким образом, разработанное приложение соответствует поставленной цели и требованиям курсовой работы. В результате была создана полноценная fullstack-система с ролевой моделью, конструктором тестов, прохождением тестирования, сохранением результатов, аналитикой, административным управлением, автоматизированным тестированием, контейнеризацией, CI/CD и размещением на хостинге.

Список используемой литературы

1. Фаулер М. Архитектура корпоративных программных приложений. - Москва: Вильямс, 2019. - 544 с.
2. Richards M., Ford N. Fundamentals of Software Architecture: An Engineering Approach. - Sebastopol: O'Reilly Media, 2020. - 432 p.
3. Fielding R. Architectural Styles and the Design of Network-based Software Architectures: Doctoral dissertation. - Irvine: University of California, 2000. - 162 p.
4. The Twelve-Factor App [Электронный ресурс]. - Режим доступа: <https://12factor.net/> (дата обращения: 26.04.2026).
5. React - A JavaScript library for building user interfaces [Электронный ресурс]. - Режим доступа: <https://react.dev/> (дата обращения: 26.04.2026).
6. React Router Documentation [Электронный ресурс]. - Режим доступа: <https://reactrouter.com/> (дата обращения: 26.04.2026).
7. Redux Toolkit Documentation [Электронный ресурс]. - Режим доступа: <https://redux-toolkit.js.org/> (дата обращения: 26.04.2026).
8. Node.js Documentation [Электронный ресурс]. - Режим доступа: <https://nodejs.org/> (дата обращения: 26.04.2026).
9. Express.js Documentation [Электронный ресурс]. - Режим доступа: <https://expressjs.com/> (дата обращения: 26.04.2026).
10. PostgreSQL Documentation [Электронный ресурс]. - Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 26.04.2026).
11. Docker Documentation [Электронный ресурс]. - Режим доступа: <https://docs.docker.com/> (дата обращения: 26.04.2026).
12. Интерактивный конструктор образовательных тестов [Электронный ресурс]: репозиторий. - Режим доступа: https://github.com/Relax1205/kursach_6_PiRKSP (дата обращения: 26.04.2026).